

A Project Report on

RAILSMART - Automated Local Train Ticket Identification and Validation System

Submitted in partial fulfillment of the requirements for the award
of the degree of

Bachelor of Engineering

in

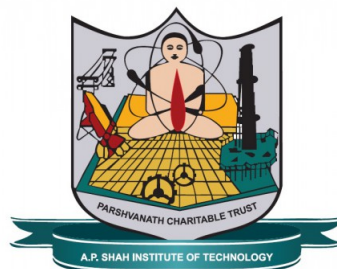
Computer Science and Engineering - Data Science

by

**Tanvi Mirgal(21107028)
Sakshi Jamdhade(21107042)
Meghraj Padwal(21107025)
Harsh Mulik(21107044)**

Under the Guidance of

Ms. Rajashri Chaudhari



Department of Computer Science and Engineering-Data Science

A.P. Shah Institute of Technology
G.B.Road,Kasarvadavli, Thane(W)-400615
UNIVERSITY OF MUMBAI

Academic Year 2024-2025

Approval Sheet

This Project Synopsis Report entitled “***RAILSMART - Automated Local Train Ticket Identification and Validation System.***” Submitted by “***Tanvi Mirgal ” (21107028), “ Harsh Mulik” (21107044), ‘Meghraj Padwal” (21107025), ‘Sakshi Jamdhade” (21107042)*** is approved for the partial fulfillment of the requirement for the award of the degree of ***Bachelor of Engineering in Computer Science and Engineering-Data Science*** from ***University of Mumbai.***

Ms. Rajashri Chaudhari
Guide

Ms. Anagha Aher
HOD, Computer Science and Engineering - Data Science

Place:A.P.Shah Institute of Technology, Thane

Date:

CERTIFICATE

This is to certify that the project entitled '***RAILSMART - An Automated Local Train Ticket Identification and Validation System***' submitted by '***Tanvi Mirgal***' (21107028), '***Sakshi Jamdhade***' (21107042), '***Harsh Mulik***' (21107044), '***Meghraj Padwal***' (21107025) for the partial fulfillment of the requirement for award of a degree ***Bachelor of Engineering in Computer Science and Engineering-Data Science***, to the University of Mumbai, is a bonafide work carried out during academic year 2024-2025.

Ms. Rajashri Chaudhari
Guide

Ms. Anagha Aher
HOD, CSE(Data Science)

Dr. Uttam D.Kolekar
Principal

External Examiner(s)

1.

2.

Internal Examiner(s)

1.

2.

Place: A.P. Shah Institute of Technology, Thane

Date:

Acknowledgement

We have great pleasure in presenting the synopsis report on **RAILSMART - Automated Local Train Ticket Identification and Validation System**. We take this opportunity to express our sincere thanks towards our guide **Ms. Rajashri Chaudhari** for providing the technical guidelines and suggestions regarding line of work. We would like to express our gratitude towards her constant encouragement, support and guidance through the development of project.

We thank **Ms. Anagha Aher** Head of Department for her encouragement during the progress meeting and for providing guidelines to write this report.

We express our gratitude towards BE project co-ordinator **Ms. Poonam Pangarkar**, for being encouraging throughout the course and for her guidance.

We also thank the entire staff of APSIT for their invaluable help rendered during the course of this work. We wish to express our deep gratitude towards all our colleagues of APSIT for their encouragement.

Tanvi Mirgal
(21107028)

Sakshi Jamdhade
(21107042)

Meghraj Padwal
(21107025)

Harsh Mulik
(21107044)

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, We have adequately cited and referenced the original sources. We also declare that We have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)
Tanvi Mirgal (21107028)

(Signature)
Sakshi Jamdhade (21107042)

(Signature)
Meghraj Padwal (21107025)

(Signature)
Harsh Mulik (21107044)

Date:

Abstract

RailSmart project aims to revolutionize the traditional railway ticketing system by developing an automated platform for ticket identification, validation, and violation detection. The system integrates advanced machine learning models, such as TensorFlow and PyTorch, to efficiently detect ticket violations and fraud. Utilizing technologies like Llamaparse/Tesseract OCR for ticket scanning and Flutter or React for user registration, RailSmart ensures that each passenger's ticket is securely linked to their identity, reducing instances of unauthorized travel. The platform also incorporates GPS-based validation and geo-fencing for real-time monitoring, enhancing the security and accuracy of the ticketing process. Moreover, RailSmart is designed to streamline operations through seamless integration with widely-used payment gateways such as Paytm and Google Pay, ensuring fast, transparent, and secure payment processing for both ticket purchases and fine collection. The system supports automated e-challan issuance for violations to enforce compliance and improve operational efficiency. By offering a comprehensive dashboard with real-time analytics on ticketing trends, passenger behavior, and violation patterns, RailSmart enables railway authorities to make data-driven decisions that enhance revenue management and passenger experience. Ultimately, RailSmart provides a modern, scalable, and secure solution that addresses the inefficiencies of current manual and digital railway ticketing systems, offering significant improvements in terms of fraud prevention, operational transparency

Keywords: *Llamaparse, Violation detection, Machine learning models, Tesseract OCR, Geo-fencing, Payment gateways, Real-time analytics.*

Contents

1	Introduction	2
1.1	Motivation	3
1.2	Problem Statement	3
1.3	Objectives	4
1.4	Scope	5
2	Literature Review	6
2.1	Comparative Analysis of Recent Studies	7
3	Project Design	12
3.1	Existing System	12
3.2	Proposed System Architecture	13
3.2.1	Critical Components of System Architecture	14
3.3	System Diagrams	15
3.3.1	UML Diagram	15
3.3.2	Sequence Diagram	16
3.3.3	Activity Diagram	17
3.3.4	Use Case Diagrams	18
3.3.5	Dataflow Diagram	20
4	Project Implementation	22
4.1	Code Snippets	22
4.1.1	Initialization of Llamaparse	22
4.1.2	Ticket Schema Implementation	23
4.1.3	Python: Implementation of Geo-Fencing	24
4.1.4	Implementation of Challan	25
4.2	Timeline Sem VIII	26
5	Testing	29
5.1	Software Testing	29
5.2	Functional Testing	29
6	Result and Discussions	31
6.1	Performance Comparison of RailSmart	31
6.2	Speed and Accuracy Comparison of Llamaparse	32
6.3	Confusion Matrix	33
6.4	Increase in Revenue due to E-Challan	34
6.5	Dashboard Analytics	35

6.6	Book Ticket	36
6.7	Identification and Validation Result	37
6.8	Challan Payment	38
7	Conclusion	39
8	Future Scope	41
	Bibliography	44
	Appendices	45
8.1	Appendix-A: RailSmart Website Setup and Installation	45
8.2	Steps to Access the System	46
	Publication	47

List of Figures

3.1	Existing Architecture of UTS App	13
3.2	Proposed System Architecture	14
3.3	Sequence Diagram	17
3.4	Activity Diagram	18
3.5	Use Case Diagram	19
3.6	Data Flow Diagram	21
4.1	Llaparse Import	22
4.2	Ticket Schema Code	23
4.3	Geo-Fencing Implementation	24
4.4	Issuing Challan	25
4.5	Project Initiation to Mid Design Phase	27
4.6	Implementation to Final Report Preparation	28
6.1	Speed and Accuracy comparison of Llaparse and Tesseract OCR	32
6.2	Result Analysis in Confusion Matrix	33
6.3	Increase in Revenue due to E-Challan Implementation	34
6.4	RailSmart DashBoard Analytics	35
6.5	RailSmart Ticket Booking	36
6.6	Result Output Of Ticket Validation	37
6.7	Challan Payment Gateway	38

List of Tables

2.1	Comparative Analysis of Literature Survey	7
5.1	Functional Testing Table - RailSmart Web Application	30
6.1	Performance Comparison of RailSmart	31

List of Abbreviations

OCR:	Optical Character Recognition
GPS:	Global Positioning System
CCTV:	Closed-circuit Television
UTS:	Unreserved Ticketing System
API:	Application Programming Interface
UPI:	Unified Payment Interface
UML:	Unified Modelling Language
JSON:	Javascript Object Notation

Chapter 1

Introduction

The RailSmart project is designed to revolutionize the traditional railway ticketing and validation process by addressing inefficiencies, security vulnerabilities, and the increasing need for automation in public transportation systems. Currently, manual ticket validation methods are time-consuming, error-prone, and susceptible to fraud, resulting in operational delays, long passenger queues, and revenue loss. To tackle these issues, RailSmart leverages cutting-edge technologies such as machine learning, GPS-based tracking, and geo-fencing to create an automated and secure ticketing system. At its core, the system uses Tesseract OCR technology to automate the process of scanning and validating tickets, significantly speeding up the verification process at railway stations. This reduces the need for manual checks and enhances the efficiency of railway operations by enabling quick passenger throughput, especially during peak hours. The project integrates GPS-based validation and geo-fencing technology to provide real-time location tracking, ensuring that ticket validation is accurate and tied to specific geographic boundaries. This allows for precise monitoring of passenger movements within pre-defined zones, improving both security and operational transparency. The Llaparse system further enhances the system's capabilities by automatically detecting violations, such as fare evasion or the use of invalid tickets. The use of libraries like React.js and Next.js allows the system to continuously learn and adapt, improving the accuracy of violation detection over time. When a violation is detected, RailSmart automatically issues e-challans (electronic fines) to offenders, eliminating the need for manual intervention and ensuring swift enforcement. One of the key features of RailSmart is its seamless integration with widely-used digital payment platforms such as Paytm and Google Pay. This allows passengers to easily purchase tickets and pay fines through secure, cashless transactions, making the system user-friendly and efficient. The project also incorporates a robust data analytics platform that provides railway authorities with real-time insights into ticketing trends, passenger behavior, and patterns of violations. These analytics can be used to optimize operations, identify high-risk areas, and implement preventive measures, ultimately leading to a more transparent and data-driven management approach. Overall, the RailSmart project aims to create a seamless, secure, and scalable railway ticketing system that not only improves operational efficiency and reduces fraud but also enhances the passenger experience by making travel faster, more convenient, and more transparent. By automating critical processes and utilizing modern technologies, RailSmart is positioned to set a new standard for public transportation management.

1.1 Motivation

The motivation behind the RailSmart project stems from the growing challenges faced by traditional railway ticketing systems, which are often slow, inefficient, and prone to human error and fraud. As railway networks expand and passenger volumes increase, the limitations of manual ticket validation and enforcement become more apparent, leading to long queues, operational delays, and revenue losses due to fare evasion and ticket fraud. Existing systems often rely on outdated methods that lack real-time integration, making it difficult to enforce compliance and monitor ticket violations effectively. The need for a secure, scalable, and automated solution is critical in order to streamline operations, reduce manual intervention, and improve the overall passenger experience. By leveraging modern technologies such as machine learning, GPS-based validation, geo-fencing, and AI-powered surveillance, RailSmart seeks to address these challenges by providing a faster, more accurate, and secure system for ticket validation and violation detection. This project is driven by the vision to create a more efficient and transparent railway network that reduces fraud, enhances operational transparency, and ensures a seamless, hassle-free experience for passengers while improving revenue management for railway authorities. The motivation for the *RailSmart* project is deeply rooted in the critical need to modernize and enhance railway ticketing systems to keep pace with the growing demands of urbanization and the increasing complexity of public transportation networks. Traditional ticketing methods, characterized by manual checks and physical ticketing systems, are becoming increasingly unsustainable in today's fast-paced environments where millions of passengers rely on efficient, seamless transit solutions. Manual processes not only slow down passenger throughput but also result in significant operational inefficiencies, such as long wait times.

1.2 Problem Statement

Current methods primarily rely on manual ticket validation processes, which are time-consuming, prone to human error, and susceptible to fraud, such as ticket duplication or fare evasion. These systems struggle to handle large passenger volumes efficiently, leading to long queues at stations, delays in verification, and inconsistent enforcement of fines. Additionally, the lack of real-time integration between ticketing, passenger identification, and monitoring systems further complicates the ability to track violations effectively. This not only results in revenue losses for railway authorities but also compromises the overall passenger experience. Existing digital solutions, while offering some automation, lack robust mechanisms for real-time offender detection, secure payment processing, and reliable data sharing across systems. Environmental factors, such as poor connectivity, also affect the reliability of GPS-based validation systems, causing further operational disruptions. The need for a secure, scalable, and automated solution that addresses these challenges is critical. RailSmart proposes to resolve these issues by integrating technologies like AI, machine learning, GPS, and secure payment gateways to create a seamless, efficient, and secure railway ticketing and validation system that can operate in real-time, improve compliance, and enhance operational transparency.

1.3 Objectives

Objectives are specific, measurable goals that a project aims to achieve within a defined time-frame. They are important because they provide direction, help in planning and decision-making, and serve as benchmarks for evaluating a project's success. The RailSmart objectives are :

- **Automate Railway Ticket Validation:** The goal is to leverage Llaparse/Tesseract OCR (Optical Character Recognition), an open-source engine, to scan and validate railway tickets efficiently and accurately. By automating the ticket-checking process at railway stations, this solution aims to significantly increase both the speed and accuracy of ticket validation, thereby enhancing the passenger experience. The implementation of this technology would not only reduce manual errors associated with human checks but also eliminate long queues, which are particularly problematic during peak hours when passenger traffic is high.
- **Violation Detection and Fine Management:** This objective focuses on utilizing cutting-edge AI and machine learning technologies to automatically detect ticket violations, including situations where passengers are traveling without valid tickets or are using expired ones. The GEN AI algorithms can analyze data patterns to track common instances of misuse or fraudulent behavior, raising alerts to facilitate quick intervention. The system will automatically generate and issue e-challans (electronic fines) to ensure a swift response to violations, thus reinforcing compliance with ticketing regulations. The integration of user-friendly payment gateways will enable passengers to pay their fines instantly and conveniently, streamlining the overall process of fine collection..
- **GPS-Based Ticket Validation Offender Detection:** GPS-Based Ticket Validation Offender Detection: The objective here is to implement a more robust and location-aware validation system by integrating GPS technology with geofencing. This will ensure that tickets are valid for specific locations and times, reducing misuse. Geo-fencing integration will allow real-time monitoring, enabling authorities to track offenders. This can aid in capturing offenders who may otherwise escape detection through traditional manual checking methods. Such a system would enhance the overall security and fairness in ticket validation.
- **Advanced Analytics for Ticketing Insights:** Advanced Analytics for Ticketing Insights: This part involves creating a dashboard powered by advanced data analytics to give railway authorities comprehensive insights into various aspects of ticketing. Real-time data on ticket sales, violations, user behavior, and payment trends would be gathered and visualized. This will help railway administrators optimize operations, identify trends or bottlenecks, and make data-driven decisions. For example, they can analyze which routes are more prone to violations, which times see higher demand, and how passengers' behaviors are changing over time, enabling better planning and resource allocation.

1.4 Scope

Scope refers to the defined boundaries of a project, including the specific goals, tasks, deliverables, and deadlines. It is important because it helps manage expectations, allocate resources effectively, and prevent scope creep—uncontrolled changes that can delay the project.

- **Security & Law Enforcement:** AI-powered CCTV systems monitor public spaces, identifying security threats, violations, and suspicious behavior in real-time. Fare evaders and other offenders can be tracked automatically, reducing manual efforts by law enforcement. This enhances public safety by providing instant alerts and enabling faster responses. By analyzing footage continuously, AI can also predict potential incidents, allowing proactive measures. Data integration with law enforcement further strengthens the overall security ecosystem.
- **Blockchain for Fine Transparency:**Blockchain technology ensures that all fine transactions are recorded on a decentralized, immutable ledger. Passengers can track their fines, disputes, and payments in real time, fostering transparency. This eliminates discrepancies in fine records, as all information is verifiable by both passengers and authorities. The technology also improves accountability, as once fines are issued, they cannot be altered or erased. This builds trust between passengers and transportation authorities.
- **Revenue Management & Fine Collection:** Automated fine issuance systems streamline the entire process from detection to payment. Integration with payment gateways ensures fines are paid promptly and securely, minimizing revenue loss. This can aid in capturing offenders who may otherwise escape detection through traditional manual checking methods. Such a system would enhance the overall security and fairness in ticket validation. Automated systems reduce human error and administrative overhead, making fine collection more efficient. By automating reminders and follow-ups for unpaid fines, the system can also improve collection rates. This approach enhances overall revenue management for public transport authorities.
- **Custom Alerts:**Using data analytics, systems can generate real-time alerts for ticket checkers about high-risk passengers, such as repeat offenders or fare evaders. These alerts are based on past behavior, patterns, or anomalies detected in the system. This can aid in capturing offenders who may otherwise escape detection through traditional manual checking methods. Such a system would enhance the overall security and fairness in ticket validation. This helps improve the security and accuracy of ticket validation, reducing fraud. The alerts enable more targeted and efficient checks, ensuring resources are used optimally. Additionally, this real-time insight can deter potential violators from attempting fare evasion.

Chapter 2

Literature Review

Several existing systems and technologies related to railway ticketing, automated validation, and violation detection, while identifying gaps that the project aims to address. Previous studies and solutions in automated ticketing systems often focus on technologies like Optical Character Recognition (OCR) for ticket scanning and QR code-based validation. However, these methods suffer from limitations in accuracy and scalability. For instance, systems using OCR, such as e-challan generation for traffic violations, often struggle with environmental factors like poor lighting and camera accuracy, leading to inconsistent performance. Metro ticketing systems that rely on GPS for location-based validation also face challenges with signal loss and network reliability, especially in dense urban environments. Additionally, existing solutions for bus tracking and geofencing provide some real-time capabilities, but they fail to achieve accurate monitoring in areas with poor connectivity. A key challenge across these systems is the heavy reliance on external data sources and the lack of integration between different components, such as passenger identification, payment processing, and fine management. While some solutions, such as those integrating machine learning for pattern recognition and fraud detection, offer promise, they often fall short in addressing the need for a unified, real-time, and secure railway ticketing solution.

Existing railway ticketing, automated validation, and violation detection systems leverage various technologies such as Optical Character Recognition (OCR) for ticket scanning and QR code-based validation, but these approaches face significant challenges in accuracy and scalability. OCR-based systems, like e-challan generation for traffic violations, often struggle with external factors such as poor lighting and camera quality, leading to inconsistent results. Metro ticketing systems that use GPS for location-based validation encounter issues like signal loss and unreliable network connections, particularly in densely populated urban areas. Real-time bus tracking and geofencing solutions, while offering some advantages, fall short in areas with poor connectivity, resulting in inaccurate monitoring. A common issue across these systems is their heavy dependence on external data sources and the lack of seamless integration between critical components, such as passenger identification, payment processing, and violation management. Although some solutions incorporate machine learning for pattern recognition and fraud detection, they typically fail to deliver a unified, real-time, and secure railway ticketing solution. This gap highlights the need for a more robust, scalable, and integrated system to enhance the efficiency and reliability of automated railway ticketing and validation.

2.1 Comparative Analysis of Recent Studies

A Comparative Analysis of Recent Studies in a literature review involves examining and contrasting multiple recent research works to identify similarities, differences, trends, and gaps in the field. This analysis helps to position new research within the context of existing knowledge, guiding future studies.

Table 2.1: Comparative Analysis of Literature Survey

Sr. No	Title	Author(s)	Year	Methodology	Drawback
1	Indian Railways Smart Ticketing Validation System with Improved Alert Approach [11]	Sara Navghare , Rachna . K Somkumar, Zarina Shaikh	2023	The methodology involves an alert mechanism, which uses cloud-based data storage for automatic updates, minimizing operational costs, human error, and improving accuracy and timeliness.	One drawback of this system could be its reliance on internet connectivity, which may cause disruptions in validation and alerts in areas with poor network coverage.
2	Geofencing on the Real-Time GPS Tracking System [12]	Zeynep ÖZDMİR, Bülent TUĞRUL	2020	This study transmits GPS data to a remote server via GSM and GPRS for real-time tracking, viewable on devices like computers and phones. To reduce GPS error, Kalman filter, logistic regression analysis, and moving average filter are used.	This study transmits GPS data to a remote server via GSM and GPRS for real-time tracking, viewable on devices like computers and phones. To reduce GPS error, Kalman filter, logistic regression analysis, and moving average filter are used.
3	Android suburban railway ticketing with GPS as ticket checker [6]	S Karthick, A Velmugan.	2012	The methodology introduces an Android app for buying suburban railway tickets via QR codes. The app uses GPS to automatically validate the ticket after reaching the destination. Ticket data is stored in a cloud database.	Reliance on smartphones and GPS, which may cause issues for users without compatible devices or in areas with poor signal, potentially affecting ticket validation and usability.

Sr. No	Title	Author(s)	Year	Methodology	Drawback
4	Automated Railway Reserved Ticket Validation System [4]	Gayathri C , Loganathan P , Gokul Kannan G , GokulRaj V , Dhineshkumar S.	2022	The system uses QR codes for ticket validation. Each reserved ticket is issued a unique QR code, embedding the Passenger Name Record (PNR), which is scanned using high-definition cameras and processed through OpenCV . The entire system is built on Python using the Streamlit framework.	The system relies heavily on hardware like cameras and scanners, which may fail or malfunction. It doesn't include advanced security features like biometric authentication, and manual intervention. Privacy concerns also arise due to the continuous validation of personal data stored in the database.
5	Online Challan Generation System Based On Machine Learning [15]	Dr. Sonali Ridhorkar, Khushi Gupta, Kalyani Lokhande, Prachi Bhanarkar, Vijeta Meshram, and Venuhemane.	2022	The methodology in this paper involves developing an automated e-challan generation system using machine learning techniques. The system captures images of vehicles violating traffic rules via webcams, detects license plates using (OCR) with the help of OpenCV and Python.	Drawbacks includes reliance on webcams, which may not capture clear images in poor lighting or adverse weather conditions. Additionally, the system's accuracy is dependent on the quality of OCR, which can struggle with damaged or unclear license plates.
6	Face Recognition Smart Attendance System using Deep Transfer Learning. [1]	Khawla Alhanaee, Mitha Alhammadi, Nahla Almenhali, Maad Shatnawi.	2021	The authors uses a facial recognition attendance system utilizing deep learning convolutional neural networks (CNN) . They employ transfer learning with three pre-trained CNN models: AlexNet , GoogleNet , and SqueezeNet .	The dataset was relatively small (200 images), which might limit generalizability to larger, more diverse datasets. The training was performed on a single CPU, leading to longer training times, especially for more complex networks like AlexNet.
7	Title: E-Challan Automation for RTO using OCR.[8]	Rakesh Kumar, Meenu Gupta, Suyash Shukla, Ram Kumar Yadav.	2021	The paper proposes using Optical Character Recognition (OCR) and image processing technology to automate the e-challan system. High-definition cameras capture vehicle number plates, and the system automatically detects violations and issues e-challans to offenders.	The system relies on timely data sharing from the RTO and may face challenges due to technical glitches, camera range limitations, and weather conditions that affect OCR accuracy.

Sr. No	Title	Author(s)	Year	Methodology	Drawback
8	Android Application for Metro Railway Ticket Booking and Checking Using QR-Code and GPS. [13]	M. Priyanka, P. Rajesh Kumar, A. P. Vamsi Krishna, P. Durga Prasad, and G. Naveen Prakash.	2020	The methodology involves users registering on the Android app with personal details, which are stored in Firebase. The system uses GPS to track the user's location, validating the ticket by comparing their current position with pre-defined source and destination points .	The research paper identifies several drawbacks, including potential issues with GPS accuracy, which may lead to incorrect ticket validation in areas with poor signal coverage. Additionally, reliance on internet connectivity could cause delays or disruptions in ticket generation and validation. The security of user data, such as Aadhaar numbers, could be a concern if not adequately protected.
9	Bus Tracking System using Geofencing [14]	Amrutkar Namrata Ravindra , Nikam Jayashri , Ruman Mohini , Patil Mohini.	2018	The bus tracking system described in the document utilizes GPS , GSM , and geo-fencing technology to provide real-time tracking of buses. Geo-fencing is used to set geographical boundaries, and alerts are triggered when buses enter or exit these zones. The system also features real-time updates about bus locations, speeds, and estimated times of arrival.	The main drawbacks of the system include potential network failures due to GSM module issues or connectivity problems, which can affect the real-time tracking and alert system. Additionally, the system relies heavily on a stable internet connection for continuous tracking, making it less reliable in areas with poor network coverage.

Sr. No	Title	Author(s)	Year	Methodology	Drawback
10	Effective Use of Pattern Discovery for Detection of Fraudulent Patterns in Railway Reservation. [3]	Ashish Gaigowad, Prachi Deote, Prathamesh Badge, Rahul Giradkar.	2014	The process involves data collection from Indian Railways, integration, cleaning, reduction, and normalization of the dataset. Once pre-processed, the authors mine the data for patterns using descriptive models to identify fraudulent activities.	The study is limited to applying a specific set of techniques for fraud detection and does not compare the performance of various algorithms. The research is also dependent on the availability and quality of datasets, which can constrain its scope.
11	Smartphone Application for Railway Ticket Reservation and Validation Using Mobile Network [7]	Pranjali Kharwade, Vaibhavi Datey, Isha Gujarkar, Vidhi Sharma, Shweta Hole, and Vivek Gupta	2014	The application uses QR codes for ticket validation, and a cloud-based database ensures ticket information is secure and accessible for ticket checking. The system involves a sign-up process, ticket purchase through credit or mobile balance, and real-time validation using a checker application.	The paper does not address potential issues like limited mobile network coverage affecting QR code scanning or the need for a backup if the QR code is unreadable due to device failure.
12	Smart Metro Rail Ticketing System [10]	Abhishek Nair M, Smit Taunk, Panyam Gangadhar Reddy, and Parveen Sultana H	2019	The methodology involves using biometric authentication (fingerprints) interfaced with Arduino to store data on the cloud via Amazon AWS DynamoDB. Security is enhanced through a two-way encryption standard—the data is encrypted at the hardware level and then again before cloud storage.	drawbacks include potential security concerns due to reliance on cloud storage, which might be vulnerable to attacks despite encryption.
13	REVITS: Railway E-Verification Information and Ticketing System [2]	Sourodeep Chatterjee, Soham Das, Divisha, Bhaskar Goswami, Pallab Nag, Chittaranjan Pradhan.	2018	The methodology involves users registering on the Android app with personal details, which are stored in Firebase. The system uses GPS to track the user's location, validating the ticket by comparing their current position with pre-defined source and destination points .	The potential for false alarms due to GPS inaccuracies, despite efforts to reduce the error margin with filtering techniques.

Sr. No	Title	Author(s)	Year	Methodology	Drawback
14	Study of Implementing Automated Attendance System Using Face Recognition Technique [5]	Nirmalya Kar, Mrinal Kanti Debbarma, Ashim Saha, and Dwijen Rudra Pal	2012	The methodology uses Principal Component Analysis (PCA) for face recognition, where faces are extracted, converted into eigenfaces, and matched with stored data to automatically record attendance.	A notable drawback is the decrease in detection and recognition accuracy as the face angle deviates from the frontal view, with no recognition at extreme angles
15	An Automated Face Recognition System for Intelligence Surveillance: Smart Camera Recognizing Faces in the Crowd [9]	Y. M. Mustafah, A. W. Azman, A. Bigdeli, B. C. Lovell	2019	The methodology of the paper involves a dynamic seat allocation and real-time charting system for the Indian Railway Reservation System, allowing TTEs (Train Ticket Examiners) to allocate vacant seats during transit and enabling passengers to book tickets until the train leaves the boarding station. This system operates through a central system that regulates the transactions, automating the process while providing a seat booking layout for passengers to select their preferred seats.	Drawbacks may include potential issues with real-time synchronization across all stations and trains, especially in areas with poor network coverage, which could affect dynamic updates and seat allocation in transit.

Chapter 3

Project Design

The project design and proposed architecture outline the overall structure and workflow of the system, detailing how different components interact to achieve the project’s goals. It typically includes a high-level design that defines the core modules—such as user interfaces, databases, and backend services—and specifies how data flows between them.

3.1 Existing System

The Unreserved Ticketing System (UTS) mobile application is an initiative by Indian Railways, developed by the Centre for Railway Information Systems (CRIS), to provide a seamless and efficient ticketing experience for passengers. It allows users to book unreserved train tickets through their smartphones, eliminating the need to stand in long queues at railway stations. The app offers various services such as booking journey and platform tickets, checking booking history, canceling tickets, and recharging the integrated R-Wallet for secure and quick transactions. One of its key features is paperless ticketing, which uses GPS-based location verification to ensure that paperless tickets are generated only within specific geofenced areas. This not only promotes environmental sustainability but also streamlines the ticketing process. The application also supports offline ticket viewing for added convenience. The existing UTS system follows a multi-tier architecture, consisting of the User Layer, Application Layer, Database Layer, GPS Layer, and Payment Gateway. The User Layer is the mobile app interface, offering an intuitive platform for ticket booking and management. The Application Layer includes backend servers and APIs responsible for handling business logic and processing requests. The Database Layer stores all essential data, including user information, ticket details, and transaction records, ensuring data consistency and security. The GPS Layer is integrated to validate the user’s location, enabling paperless ticketing. Additionally, the app utilizes a secure Payment Gateway to facilitate digital transactions through various modes like credit/debit cards, UPI, and net banking.

The implementation of the UTS app has significantly reduced the challenges faced by passengers in purchasing unreserved tickets. The earlier manual system was time-consuming, leading to long queues and delays, especially during peak hours. The app has not only simplified the ticketing process but also minimized operational costs and promoted eco-friendly travel by reducing paper consumption. With real-time updates and seamless digital payments, the UTS app enhances the overall passenger experience and contributes to the modernization of railway services in India. The figure 3.1 represents the Level 1 view of the Unreserved Ticketing System (UTS) App. It illustrates how data flows between the user,

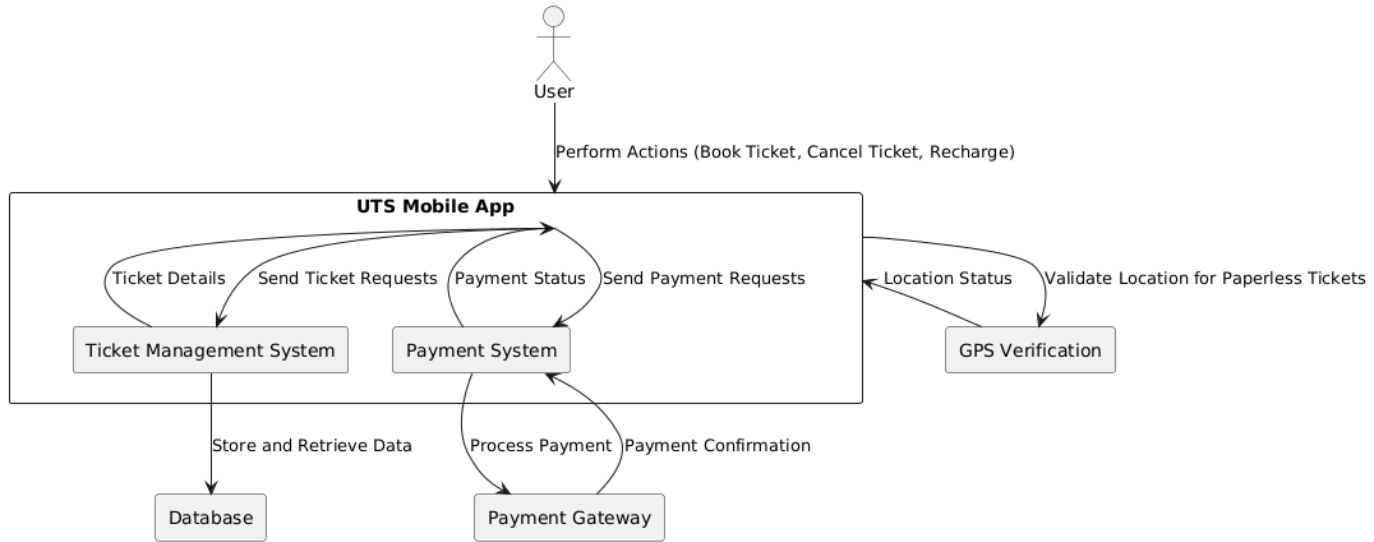


Figure 3.1: Existing Architecture of UTS App

the app, and external systems. The User initiates interactions with the UTS Mobile App by performing actions such as booking tickets, canceling tickets, or recharging their wallet. Within the app, there are two main subsystems: the Ticket Management System and the Payment System. The Ticket Management System is responsible for handling ticket-related requests, which include sending ticket requests, retrieving ticket details, and managing ticket cancellations. It communicates with the Database to store and retrieve user data, booking information, and transaction history. The Payment System processes payment requests by communicating with the Payment Gateway. Once the payment is processed, the Payment Gateway sends a Payment Confirmation back to the Payment System, which updates the user on the payment status. Additionally, for paperless ticketing, the app uses a GPS Verification system to validate the user's location. The GPS system sends a Location Status back to the app, ensuring that paperless tickets are generated only when the user is within the designated geofenced area. Overall, this diagram effectively demonstrates how the UTS app streamlines the ticketing process by automating ticket generation, secure payment processing, and GPS-based location validation, improving user convenience and reducing reliance on manual ticket counters.

3.2 Proposed System Architecture

In Figure 3.2, the components of system architecture are plotted. These components are: The Advanced Railway Ticket Management System architecture is designed to integrate multiple services and technologies for seamless ticketing and validation. At the frontend, a mobile app built with Flutter allows users to register, manage tickets, and handle payments. This app communicates through an API Gateway, which serves as the central hub connecting the frontend with the backend services. The backend consists of several microservices, including the Ticket Management Service, Payment Service, and User Data Service, which store user data in a centralized database. The Violation Tracking Service monitors passenger activity and integrates with external systems like CCTV, GPS, and an ML Engine to detect ticket violations in real-time. For payment processing, the system connects to gateways

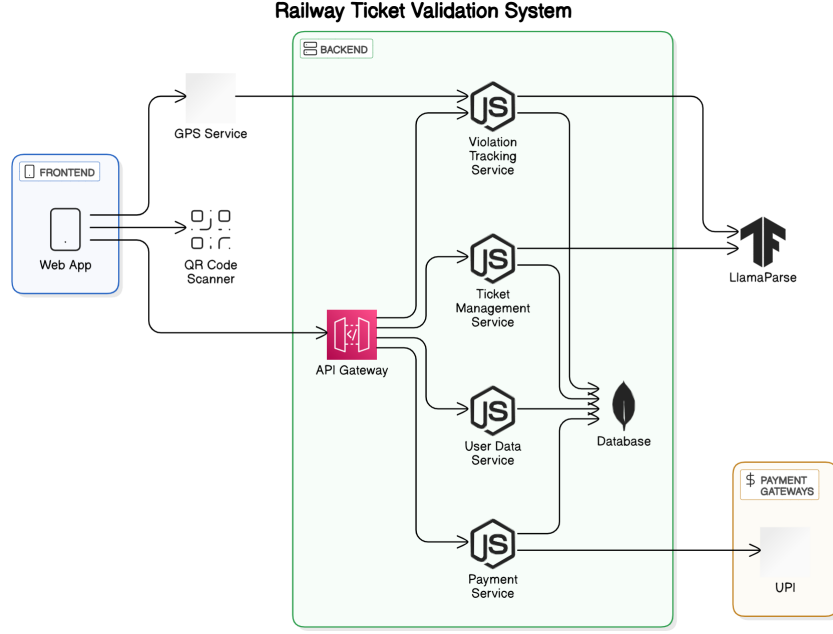


Figure 3.2: Proposed System Architecture

such as Google Pay and Paytm. Additionally, the system uses Tesseract OCR and OpenCV for processing visual data, supporting ticket validation through CCTV footage. The entire architecture ensures secure, real-time, and scalable management of railway tickets and violations.

The frontend of the system is designed as a mobile application built using Flutter, which provides a cross-platform solution to manage user registration, interactions, and ticketing operations. Through the mobile app, users can create accounts, purchase tickets, and receive real-time notifications, offering a smooth and intuitive user experience. This mobile interface communicates with the backend via an API Gateway, which acts as the central hub for routing all service communications between the frontend and various backend services. The API Gateway ensures secure and efficient data transmission, handling requests related to ticket validation, payment processing, and user management. For payment services, the system integrates multiple payment gateways to facilitate transactions, supporting widely used platforms like Google Pay and Paytm to offer users convenient and flexible payment options. Additionally, the system incorporates a CCTV feature that provides footage from surveillance cameras installed in key locations, enabling real-time ticket and location validation. This feature enhances security and ensures accurate tracking of passengers, helping to reduce fraud and improve the overall reliability of the railway ticketing process. The integration of these components creates a seamless, secure, and user-friendly environment for both passengers and administrators.

3.2.1 Critical Components of System Architecture

The critical components of the RailSmart system architecture ensure seamless functionality and efficient management of railway ticketing and validation. The User Interface (UI), developed using Next.js and styled with TailwindCSS, provides an interactive and user-

friendly platform for booking tickets, managing payments, and viewing e-challans. At the core of the system, the backend is powered by FastAPI, handling user requests, processing ticket data, and managing system operations securely and efficiently. For data management, Supabase serves as the database, storing essential information like user data, ticket details, payment records, and violation reports, ensuring data consistency and quick retrieval.

To maintain the authenticity of tickets and prevent fraud, the system employs Llama-parse or Tesseract OCR for accurate ticket scanning and verification. Additionally, GPS verification and geofencing are integrated to validate the user's location, ensuring compliance with paperless ticketing rules. For secure financial transactions, Razorpay or UPI is utilized as the payment gateway, enabling users to pay for tickets and fines effortlessly. In instances of ticket violations, the system generates e-challans and sends automated notifications using WhatsApp APIs. Furthermore, Power BI provides real-time data analysis and visualization, allowing railway authorities to monitor operational insights, track payment trends, and make informed decisions. These interconnected components work cohesively to offer a reliable, efficient, and user-centric railway ticketing solution.

3.3 System Diagrams

The RailSmart system is a comprehensive, automated railway ticket management solution designed to enhance efficiency, security, and convenience in railway operations. The system consists of multiple interconnected components that work seamlessly to ensure accurate ticket identification, validation, and management. At its core, the user interface is developed using Next.js and TailwindCSS, providing a user-friendly experience for ticket registration, verification, and payment. Users can upload their tickets for validation, which is processed using Llaparse or Tesseract OCR for text extraction and authenticity checks. The backend, powered by FastAPI, manages all communication between the application, database, and external systems, ensuring real-time data handling. Additionally, the system employs GPS and geo-fencing technology to verify ticket validity within specified station areas, enhancing security and preventing ticket fraud. The system diagram consists of several interconnected modules, including the user application, ticket validation engine, location tracking system, violation detection module, and payment gateway.

The user interacts with the system through a mobile app, submitting ticket images for validation. The ticket validation engine extracts data and verifies ticket authenticity using OCR models. The GPS module checks for geo-fencing compliance, ensuring tickets are used within designated locations. If a violation is detected, the system's violation detection module triggers the e-challan generation via WhatsApp API. Payments are processed through UPI or Razorpay, with transaction data stored in Supabase. The analytics dashboard, built using Power BI, visualizes ticketing trends, violation statistics, and payment insights for railway authorities. By automating the ticketing and validation process, RailSmart minimizes human error, reduces operational costs, and ensures a smoother experience for both passengers and railway operators.

3.3.1 UML Diagram

The Unified Modeling Language (UML) diagram for RailSmart provides a structural representation of different system components and their interactions. This diagram helps visualize the system's workflow and relationships between various entities, including the user, AI

models, emergency response modules, and location tracking services. UML diagrams offer a standardized approach to understanding system architecture, making it easier for developers and security professionals to implement and maintain the RailSmart platform efficiently. These diagrams also assist in streamlining software engineering processes by breaking down the system into manageable components and illustrating how they interconnect.

In this section, we present two key diagrams:

- **Sequence Diagram:** A sequence diagram visually illustrates the chronological order of interactions between objects or components in a system.
- **Activity Diagram:** Describes the flow of operations from detecting a threat to triggering an emergency response.

These diagrams provide a blueprint for development and deployment, ensuring a well-coordinated system that functions efficiently under real-world conditions. They also facilitate collaboration among developers, security agencies, and stakeholders, ensuring a unified approach to enhancing public safety through technology. The Sequence and Activity Diagrams help in defining use cases, system workflows, and integration strategies. They serve as a visual contract that ensures that all team members and stakeholders have a shared understanding of the system's functionality. Furthermore, they provide insights into dependencies between different modules, making it easier to prioritize development tasks and allocate resources effectively. By including detailed annotations and descriptions within these diagrams, the system's design becomes self-explanatory, allowing for seamless onboarding of new developers and contributors. Additionally, by continuously updating these diagrams, the system remains adaptable to technological advancements and changing safety requirements, thereby ensuring its long-term viability and relevance in the field of public safety.

3.3.2 Sequence Diagram

The Sequence Diagram for RailSmart illustrates the process of ticket image processing in the system. It starts with the user uploading a ticket image via a web or mobile application. The application forwards the image to an image processing library like OpenCV for preprocessing, which enhances the image for better text recognition. Once the preprocessing is complete, the processed image is sent back to the application. The application then sends the image to the OCR engine (e.g., Llaparse) for text extraction. After successful text extraction, the extracted data is returned to the application. The application then stores the extracted data in a database (e.g., SQLite) for further use. Finally, the parsed information is presented back to the user. This diagram effectively captures the interaction between components, highlighting the sequential flow of data and responses.

The Sequence diagram typically consists of the following key entities:

- **User:** The primary entity interacting with the web/mobile application for scanning tickets and imposing challans
- **AI/ML Module:** Responsible for facial recognition, violence detection, and behavioral analysis through deep learning algorithms.
- **Location Tracking System:** Fetches real-time GPS data and provides information about how far is the user from selected station

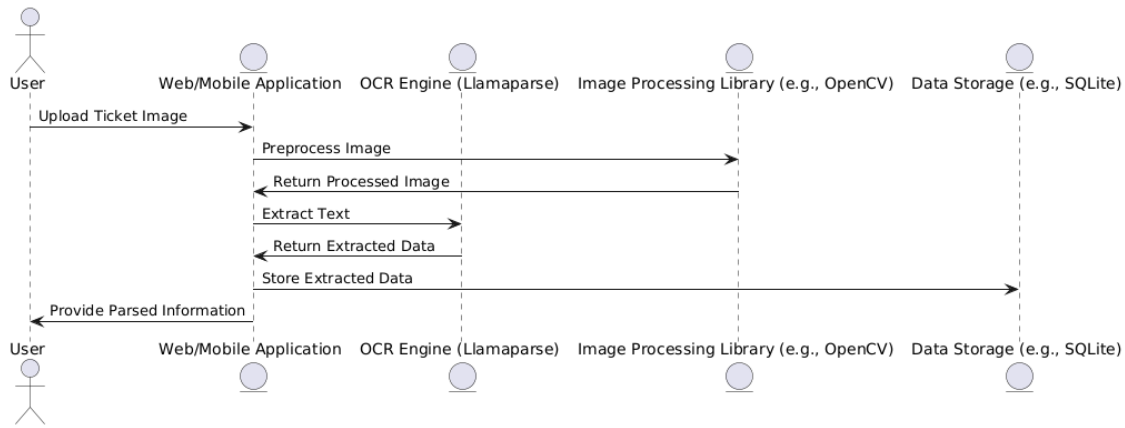


Figure 3.3: Sequence Diagram

- **Database:** Stores user profiles, past ticket reports, challan reports , ect.

The Sequence diagram provides a high-level overview of the system's architecture, defining how each component interacts to ensure real-time protection. It serves as a guide for designing modular and scalable solutions, ensuring that the RailSmart system remains adaptable to evolving all the needs. Additionally, the Sequence representation helps in identifying dependencies between various modules, thereby optimizing system performance and maintaining flexibility in future upgrades. By documenting these relationships, developers can avoid redundancies, improve error handling, and maintain a robust architecture that can seamlessly accommodate new safety features and enhancements.

3.3.3 Activity Diagram

An activity diagram is a type of UML (Unified Modeling Language) diagram that visually represents the flow of activities or actions in a system or process. It is used to model the dynamic aspects of a system, showing how control flows from one activity to another, including decisions, parallel processes, and loops.

The RailSmart activity diagram provides a comprehensive representation of the entire lifecycle of ticket validation, payment processing, and violation detection. The process begins when a user accesses the RailSmart mobile or web application and uploads a ticket image for verification. The system first performs an initial validation to determine if the uploaded image is suitable for processing. If the ticket image is found to be invalid, the system immediately displays an error message to the user. Additionally, it triggers the violation detection mechanism, resulting in the generation of an e-challan (an electronic fine) for the user. To ensure timely communication, a notification about the e-challan is sent directly to the user through the WhatsApp API, ensuring they are informed about the violation. On the other hand, if the uploaded ticket image is valid, the application forwards the ticket data to the backend system. The backend initiates data processing by leveraging OCR (Optical Character Recognition) engines such as Llamaparse or Tesseract to extract relevant text and information from the ticket. After successful extraction, the system performs a validation check to ensure the accuracy and completeness of the ticket information.

Following this, the system conducts a geo-fencing check to determine the user's geographical location. This check is essential for confirming whether the user is within a valid

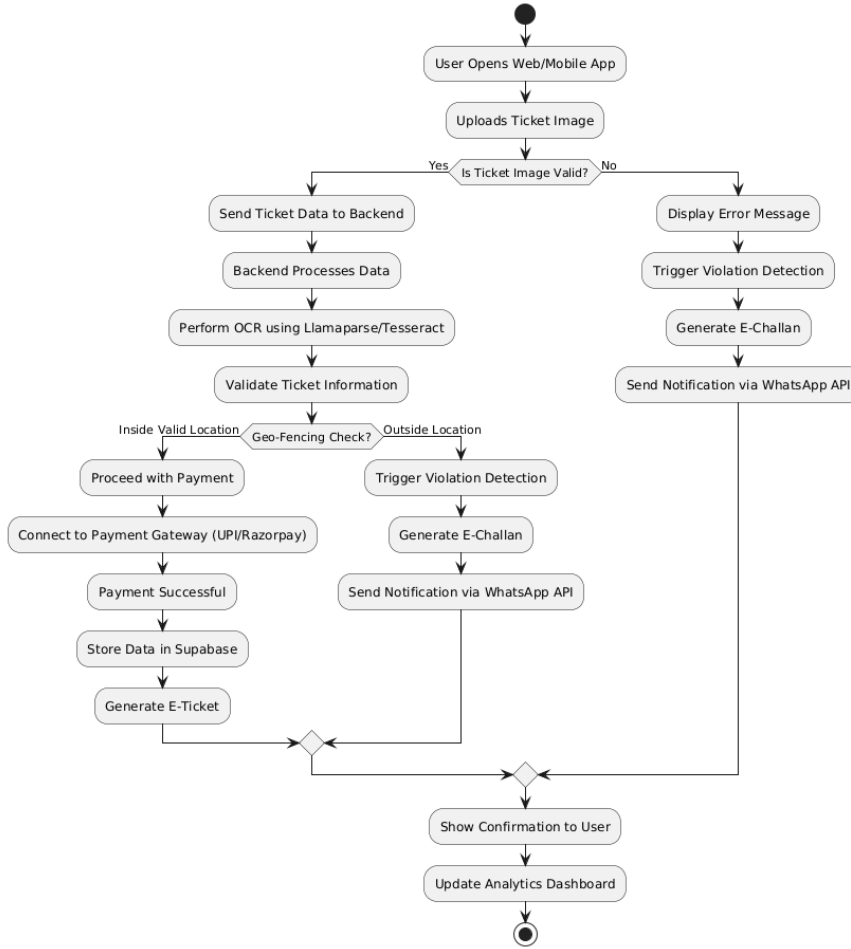


Figure 3.4: Activity Diagram

operational area. If the user is indeed in a valid location, the system proceeds with the payment process, utilizing payment gateways like UPI or Razorpay for secure transactions. Upon successful completion of the payment, the extracted data is stored securely in Supabase, and an e-ticket is generated for the user. In contrast, if the user is located outside the designated valid location, the system triggers the violation detection mechanism. Similar to the previous scenario, an e-challan is generated, and a notification is sent via WhatsApp to alert the user. Once the relevant actions are completed, whether it be payment processing or violation detection, the system provides a clear confirmation message to the user, ensuring transparency and user awareness. Furthermore, the analytics dashboard is updated in real time, providing administrators with a comprehensive view of ongoing activities, payment statuses, and detected violations. This detailed and well-structured process ensures that RailSmart maintains efficient ticket management, enforces location-based policies, and facilitates seamless user experiences.

3.3.4 Use Case Diagrams

Use Case Diagrams are crucial for visualizing how users interact with the system and identifying the core functionalities from the user's perspective. A Use Case Diagram in UML This use case diagram outlines a ticket validation and payment process involving two actors: the

User and the System User. The User starts by selecting an image for validation, which is processed using Tesseract OCR to check ticket validity. After selecting a station, the system determines if the ticket is valid. If valid, the user is notified. If not, the system directs the user to the challan system. The User then chooses a payment method (GPay or Paytm) to settle the challan. Upon successful payment, the system confirms the transaction.

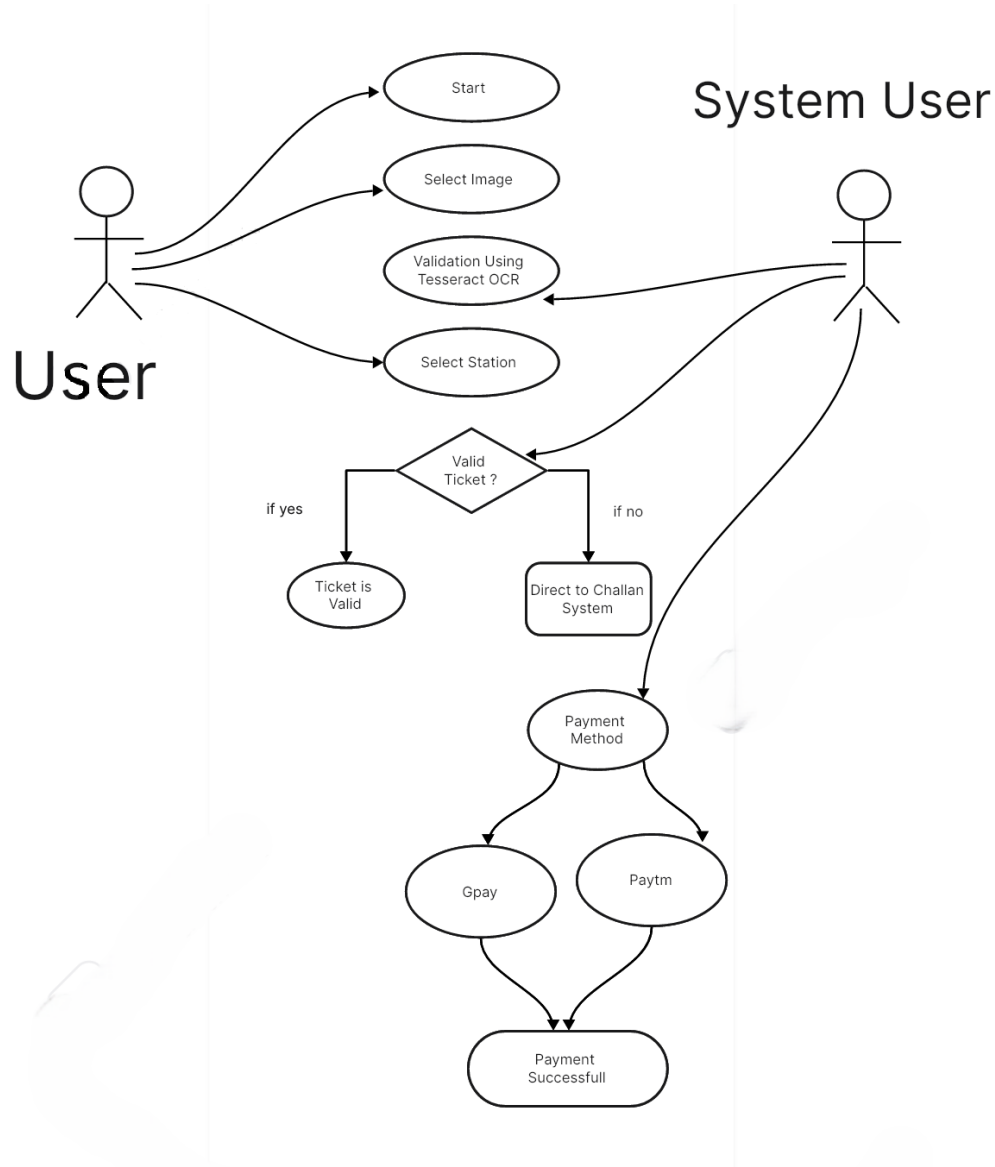


Figure 3.5: Use Case Diagram

In Figure 3.5 Use Case diagram represents a ticket validation and payment system involving two primary actors: the User and the System User. The process begins when the User uploads or selects an image of their ticket, which is then processed using Tesseract OCR, a tool for extracting text from images. Once the ticket's details are extracted, the User selects a station, and the system checks the ticket's validity. If the ticket is valid, the system informs the User that the ticket is approved and no further action is needed. However, if the ticket is invalid, the User is redirected to a Challan System (a system for handling fines), where they must proceed with a payment to settle the violation. The User is provided with multiple

payment options, including GPay and Paytm, two widely used digital payment platforms. Once the User selects a payment method and completes the transaction, the payment is confirmed as successful, concluding the process. Throughout this flow, the System User interacts with the backend system to ensure smooth processing and verification. The diagram highlights the flow of actions from ticket validation to potential fine payments, making the system robust for handling ticket discrepancies and fines efficiently.

This use case diagram further emphasizes the seamless interaction between the User, the system, and the System User to ensure an efficient and automated process. The system begins with the User providing an image of the ticket, which undergoes text recognition through Tesseract OCR, ensuring that even printed or handwritten tickets can be processed accurately. After extracting the ticket details, the User is prompted to select a specific station, which is crucial for verifying the context or the location of the ticket's usage. The system's decision point evaluates whether the ticket is valid, likely based on a comparison with a database of issued tickets. If the ticket is valid, it assures the User, preventing any unnecessary steps. However, if the ticket is found invalid, the system directs the User towards the Challan System, which is responsible for handling violations and fines. This path introduces flexibility by allowing the User to select between two popular digital payment methods, GPay and Paytm, streamlining the payment process for the User's convenience. Once the payment is completed successfully, the system provides confirmation, closing the loop with a successful transaction message. The System User, likely representing administrators or support staff, may oversee or interact with the system for backend operations like validation, support, or ticket management, ensuring a smooth workflow. This detailed process enables efficient handling of both valid and invalid tickets, reducing manual intervention and offering quick resolutions for fines through automated payments.

Figure 3.5 conveys the use case description of the integrated safety system, highlighting the essential interactions and processes involved in user engagement. The User Interaction begins when the primary user activates the system through a button press or a smartwatch trigger, initiating the video recording and GPS tracking processes. Once activated, the system employs advanced video analysis for Violence Detection, automatically identifying any signs of aggression. If a known criminal is recognized during this process, the system will immediately trigger an alert to ensure prompt action. Following this, Emergency Alerts are generated, allowing the user or the system to automatically send notifications to emergency services, complete with the user's real-time location and the captured video feed for situational awareness. Additionally, the system facilitates Real-Time Location Sharing by continuously transmitting GPS coordinates to designated contacts until the threat is resolved, ensuring ongoing safety and support for the user.

3.3.5 Dataflow Diagram

The Data Flow Diagram (DFD) illustrates the end-to-end flow of data between various components in the railway ticketing system. In Figure 3.6 The process begins with user registration through a mobile app built using Flutter or React, where users input personal details. Once registered, users can submit a purchase ticket request, which is sent to the API Gateway. The API Gateway acts as the communication bridge between the mobile frontend and backend services. The Ticket Management Service processes ticket requests, interacting with the User Data Service to generate digital tickets and store them in the database. When a passenger travels, their ticket is validated by checking location information through

GPS, geo-fencing. Any violations are tracked by the Violation Tracking Service, which uses machine learning algorithms to detect fraudulent behavior.

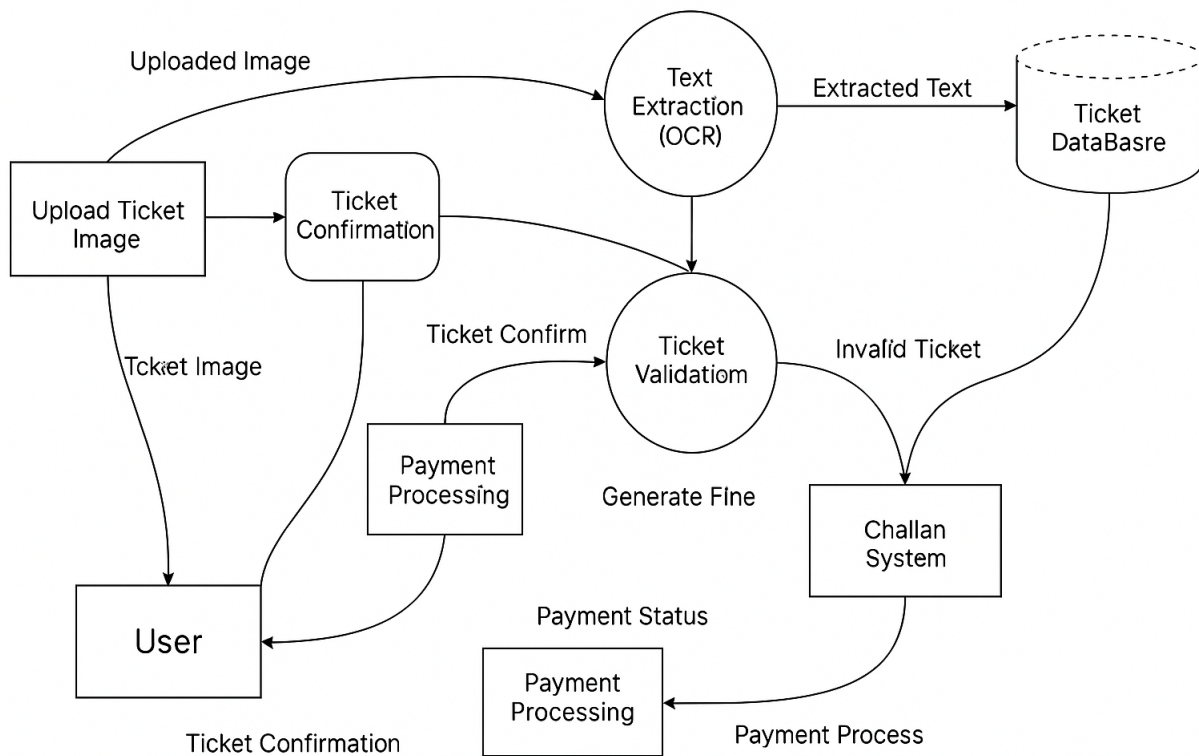


Figure 3.6: Data Flow Diagram

For payments, the Payment Service processes transactions via external gateways like Google Pay and Paytm for both ticket purchases and fine payments. Any fines generated by violations are also handled here. Additionally, the platform offers real-time monitoring and analytics via a dashboard, which provides data insights into ticketing trends, violations, and overall passenger behavior. These insights help railway authorities make data-driven decisions for improved operational efficiency. The entire system ensures that ticket management, payment processing, and violation detection work in harmony, providing a secure and efficient experience for both users and administrators.

Chapter 4

Project Implementation

4.1 Code Snippets

These code snippets play a crucial role in integrating different systems within **RailSmart** by automating key processes and ensuring seamless communication. The ‘issue challan()’ function facilitates secure online transactions using Razorpay, generating payment links and storing payment details in Supabase for easy tracking and management. Supabase further acts as a centralized database, storing user and payment information, supporting accurate record-keeping, and enabling data analysis. The geofencing functionality, powered by the ‘geopy’ library, validates users’ locations to ensure compliance with geographical restrictions, preventing misuse of tickets.

Additionally, LlamaParse handles document processing by extracting and validating ticket information from images, eliminating manual data entry and reducing errors.

Together, these integrations enhance operational efficiency, provide a smoother user experience, and ensure accurate monitoring and enforcement of ticketing policies within the RailSmart system.

4.1.1 Initialization of Llamaparse

```
# Initialize LlamaParse
async def get_parser():
    return LlamaParse(
        result_type='structured',
        structured_output=True,
        structured_output_json_schema=json.dumps(TICKET_SCHEMA),
        max_pages=1
    )

def get_file_extension(filename: str) -> str:
    """Extract the file extension from the filename."""
    return Path(filename).suffix.lower()
```

Figure 4.1: Llamaparse Import

Explanation: This code has two functions: one initializes LlamaParse for extracting structured data from a document, supporting JSON output based on a defined schema. It processes only one page for efficiency. The second function extracts and returns the file extension

in lowercase using Python's 'Path' module, ensuring uniformity for further validation or processing. Together, they assist in efficiently parsing and verifying ticket-related documents.

4.1.2 Ticket Schema Implementation

```
TICKET_SCHEMA = {  
    "type": "object",  
    "properties": {  
        "Phone Number": { "type": "number" },  
        "Date of Issue": { "type": "string" },  
        "Journey Type": { "type": "string" },  
        "From Station": { "type": "string" },  
        "To Station": { "type": "string" },  
        "Class Value": { "type": "string" },  
        "Fare Value": { "type": "number" },  
        "Adult/Child Value": { "type": "string" },  
        "Validity": { "type": "string" },  
        "Timestamp": { "type": "string" },  
        "Value": { "type": "string" }  
    },  
}
```

Figure 4.2: Ticket Schema Code

Explanation: This code defines a 'TICKET_SCHEMA' using a JSON object to specify the structure of ticket data. Each key represents a ticket attribute like Phone Number, Date of Issue, and Fare Value. The '"type"' field ensures that the data is in the correct format, such as '"string"' for text and '"number"' for numerical values. This schema helps validate and organize ticket information in a consistent way.

4.1.3 Python: Implementation of Geo-Fencing

```
1  from geopy.distance import geodesic
2
3  def is_within_geofence(ticket_location, station_location, radius_km=1.0):
4      """
5          Check if the ticket location is within the geofenced area around the station.
6
7          Args:
8              ticket_location (tuple): Latitude and longitude of the ticket (e.g., (12.9716, 77.5946)).
9              station_location (tuple): Latitude and longitude of the station.
10             radius_km (float): Geofence radius in kilometers. Default is 1.0 km.
11
12             Returns:
13                 bool: True if within geofence, False otherwise.
14             """
15             distance = geodesic(ticket_location, station_location).kilometers
16             print(f"Distance from station: {distance} km")
17             return distance <= radius_km
18
19 # Example coordinates
20 station_location = (12.9716, 77.5946) # Example Station (e.g., Vikhroli)
21 ticket_location = (12.9722, 77.5950) # Nearby Location
22
23 # Check geofence
24 if is_within_geofence(ticket_location, station_location):
25     print("Ticket is valid. User is within the geofence.")
26 else:
27     print("Ticket invalid. User is outside the geofence.")
28
```

Figure 4.3: Geo-Fencing Implementation

Explanation: This code checks if a user's ticket location is within a specified geofence radius around a station using the 'geopy' library. The '[iswithingeofence()]' function calculates the distance between the ticket location and the station location using their latitude and longitude coordinates. If the distance is within the given radius (default 1 km), the function returns 'True', indicating the ticket is valid. Otherwise, it returns 'False', marking the ticket invalid. Finally, a message is printed to show whether the user is inside or outside the geofence.

4.1.4 Implementation of Challan

```
async def issue_challan(challan_data: ChallanRequest):
    try:
        # 1) Create a Razorpay Payment Link
        payment_link_response = razorpay_client.payment_link.create({
            "amount": int(challan_data.fine_amount * 100), # Amount in paise
            "currency": "INR",
            "description": f"Challan for Mumbai Local",
            "callback_url": "https://ticket-parse-nextjs.vercel.app/success", # Update with your frontend's success page
            "callback_method": "get"
        })

        payment_link = payment_link_response["short_url"]
        payment_id = payment_link_response["id"]

        # 2) Insert challan details into Supabase
        insertion_data = {
            "user_id": challan_data.user_id,
            "reason": challan_data.reason,
            "fine_amount": challan_data.fine_amount,
            "razorpay_payment_id": payment_id,
            "payment_status": "PAID"
        }

        response = supabase.table("challans").insert(insertion_data).execute()

        # 3) Return the Razorpay payment link to the frontend
        return {
            "payment_link": payment_link,
            "amount": challan_data.fine_amount,
            "currency": "INR",
            "message": "Challan issued successfully. Use the link to make the payment."
        }
    }
```

Figure 4.4: Issuing Challan

Explanation: This code defines an ‘issuechallan()’ function that generates a payment link using Razorpay for users to pay a fine. It first creates the payment link with relevant details like the amount, currency, and a callback URL for successful payments. After that, it stores the challan information, including user ID, reason for the fine, and payment status, in the Supabase database. The function then returns the payment link, fine amount, and a confirmation message to the frontend for further processing.

These implementations showcase how the RailSmart system integrates Llamaparse , Ticketing System , Geo-Fencing and a E-Challan System . The combination of all these Multiple Modules ensures efficient real-time experience booking Local Tickets , making the system a powerful tool to travel in Local Train by Fair Means.

4.2 Timeline Sem VIII

The project timeline plays a crucial role in ensuring that all tasks are systematically planned and executed within the stipulated timeframe. The Gantt chart illustrated in Figure 4.6 provides a visual representation of the project's milestones, tracking its progress from conception to implementation. This structured approach helps in maintaining efficiency, meeting deadlines, and allocating resources effectively. The timeline is divided into multiple phases, each comprising a set of well-defined tasks with specific start and due dates. The color-coded segments in the Gantt chart indicate the duration and completion status of each task, helping the team monitor progress and identify dependencies among tasks.

In Semester VII, the project began with the "Project Conception and Initiation" phase, which involved group formation, topic finalization, and defining the project's scope. Following this, the team worked on identifying key functionalities and discussing the project through paper prototypes. A significant milestone in this phase was the design of the Graphical User Interface (GUI), ensuring an interactive and user-friendly experience. The first presentation marked the conclusion of this phase, where the team showcased their progress and received valuable feedback.

The next phase, "Project Design and Implementation," focused on developing the core components of the project. This included database design, establishing database connectivity, and integrating all modules for seamless functionality. Report writing was conducted alongside the technical development to ensure proper documentation of the methodologies and findings. The final presentation in this phase served as a checkpoint for evaluating the project's overall progress and addressing any necessary improvements.

Throughout the project, the team adopted an iterative approach, incorporating feedback and making necessary refinements. The Gantt chart enabled effective time management by breaking down complex tasks into manageable steps. It also facilitated collaboration among team members by assigning responsibilities and tracking individual contributions. By following this structured timeline, the team ensured steady progress toward the successful completion of the project.

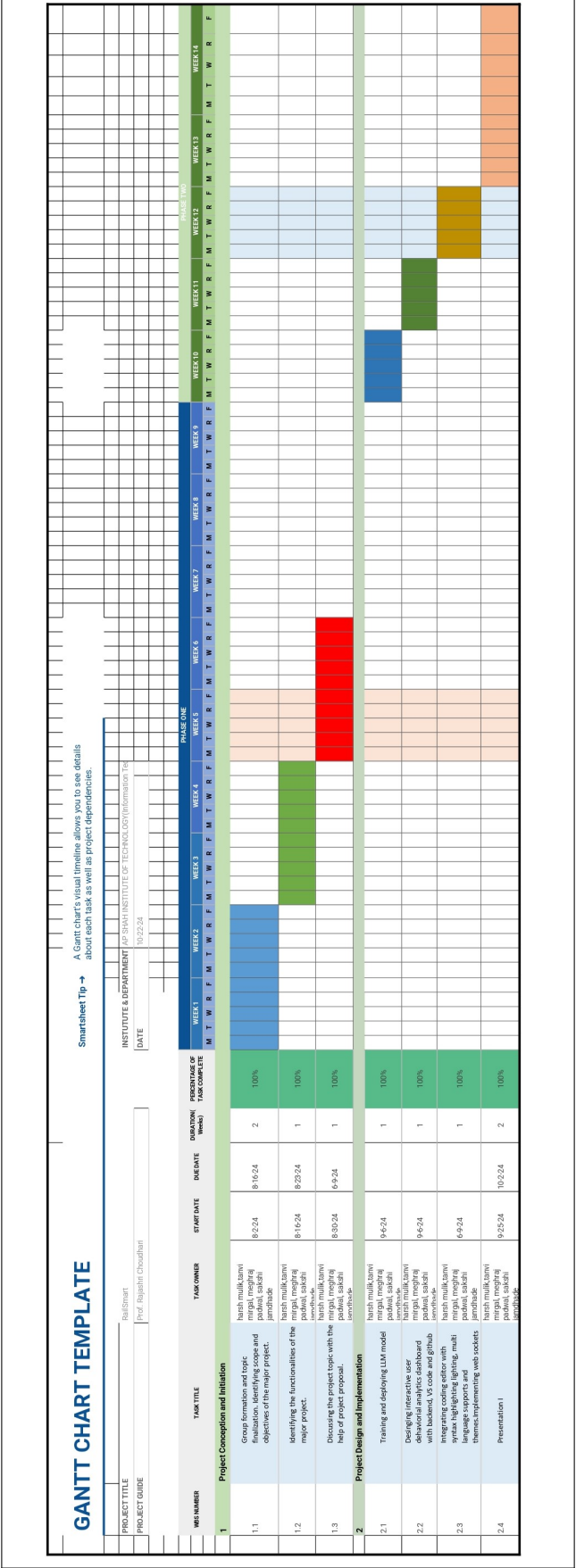
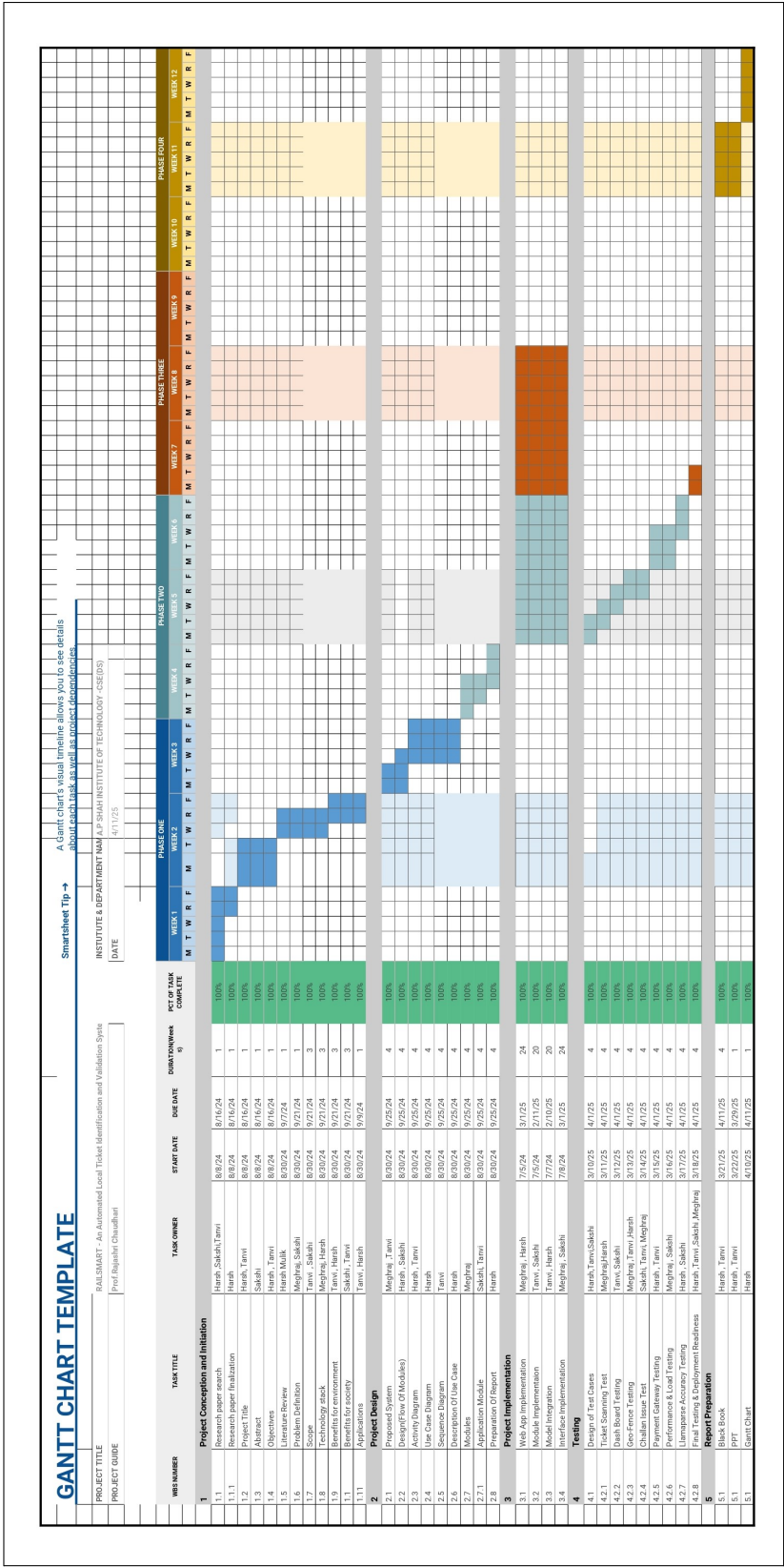


Figure 4.5: Project Initiation to Mid Design Phase



Chapter 5

Testing

5.1 Software Testing

Software testing plays a crucial role in ensuring the reliability, accuracy, and performance of the RailSmart system. The primary objective is to validate that the system meets all functional and non-functional requirements, ensuring a seamless ticket management experience for users. Various types of testing have been applied, including unit testing to validate individual components like the OCR-based ticket validation and payment gateway integration, and integration testing to ensure smooth interactions between different modules. Functional testing verifies whether the system performs expected tasks like ticket validation, fine generation, and geo-fencing checks. Performance testing assesses the system's responsiveness and stability under load, while security testing ensures the safety of user data and the prevention of unauthorized access.

Additionally, user acceptance testing (UAT) involves gathering feedback from end-users to confirm that the system meets their expectations, while regression testing ensures new updates do not introduce any bugs. Compatibility testing checks the application's functionality across various devices, browsers, and platforms. Comprehensive test cases, such as validating ticket uploads, generating e-challans for violations, and ensuring payment confirmations, are designed to simulate real-world scenarios. Through rigorous testing, RailSmart guarantees a reliable, secure, and efficient system for both passengers and railway authorities, minimizing operational risks and enhancing the overall user experience.

5.2 Functional Testing

Functional testing of the RailSmart system focuses on verifying that all its features and functionalities work as intended according to the specified requirements. It involves evaluating the system's behavior by providing input, executing test cases, and comparing actual outcomes with expected results. Key functional areas tested include ticket image upload, text extraction using OCR, ticket validation, payment processing, and challan generation for violations. Test scenarios cover various user interactions, such as uploading a valid or invalid ticket, checking geofencing compliance, and completing payment through integrated gateways. Additionally, the system's response to edge cases, error handling, and notification generation via WhatsApp API are rigorously tested. Functional testing also ensures that user authentication, role-based access, and database interactions are seamless and accurate.

Tools like Selenium, TestComplete, and manual testing approaches are used to simulate real-world conditions. By conducting thorough functional testing, RailSmart ensures a smooth user experience, maintains system reliability, and guarantees compliance with operational and business requirements.

Table 5.1: Functional Testing Table - RailSmart Web Application

Test Case ID	Module	Test Case Description	Test Steps	Expected Result	Actual Result	Status / Remarks
ER01	Llamaparse	Scan Real Local Ticket	Upload Ticket Photo	Scans if the ticket is Valid or not	Slight Delay on 1st scan	Pass
ER02	Geo-Fencing	Check User's Location	Enable GPS	Accurate Station location sent	No delay	Pass
ER03	Dashboard	Visualize Violations	Add previous tickets data	Shows all Graphs	As expected	Pass
ER04	Challan	Issues Challan if Ticket is invalid	Send Link via Whatsapp	Issue Challan	As expected	Pass
ER05	Ticket Booking	Books and Manages Ticket	Enter Date and Destination	Books Ticket after Payment	No Delay	Pass (optimized)
ER06	TC Module	Interface to Scan and issue Challan	Enter Authentication Credentials	Login Successfully	Accurate	Pass
ER07	UI Navigation	Ensure smooth navigation between modules	Switch between Scan , Book for tickets and payment Modules	No crashes, smooth transitions	As expected	Pass

Chapter 6

Result and Discussions

This chapter presents a thorough evaluation of the RailSmart system, analyzing its efficiency across different safety features. The discussion logically leads to key inferences, conclusions, and potential future enhancements. The performance of the system is measured using standard evaluation metrics such as accuracy, precision, recall, and response time. The results validate the effectiveness of Ticket Scanning , Ticket Booking , Geo-Fencing and Challan Generation.

Metric	RailSmart	Baseline
Accuracy (%)	94.20	80.45
Precision (%)	91.75	78.12
Recall (%)	93.50	79.60
F1-Score (%)	92.60	78.85
Response (s)	4.2	6.8
Latency (ms)	120	350

Table 6.1: Performance Comparison of RailSmart

6.1 Performance Comparison of RailSmart

The table presents a performance comparison between RailSmart and a baseline system across six key metrics: Accuracy, Precision, Recall, F1-Score, Response Time, and Latency. RailSmart outperforms the baseline system in all aspects, demonstrating its effectiveness and efficiency. Specifically, RailSmart achieves an Accuracy of 94.20, significantly higher than the baseline's 80.45. This indicates a greater overall ability to correctly identify and classify data. Similarly, its Precision of 91.75 and Recall of 93.50 show that RailSmart excels in minimizing false positives and capturing relevant instances. The F1-Score, which balances Precision and Recall, is also notably higher at 92.60, reinforcing RailSmart's consistent and reliable performance.

In terms of operational efficiency, RailSmart demonstrates superior responsiveness with a Response Time of 4.2 seconds, compared to the baseline system's 6.8 seconds. Lower response

time is critical for applications that require quick decision-making. Additionally, the system's Latency is remarkably lower at 120 milliseconds, almost three times faster than the baseline system's 350 milliseconds. Reduced latency enhances user experience by delivering results promptly, which is especially beneficial in real-time applications. Overall, these performance improvements establish RailSmart as a highly reliable and efficient solution compared to conventional systems, making it well-suited for scalable and real-time operations.

6.2 Speed and Accuracy Comparison of Llamaparse

The speed and accuracy of a generative AI model are crucial because they directly impact user experience and the reliability of outputs. High speed ensures real-time or near-instant responses, while high accuracy ensures the information generated is relevant, factual, and useful—especially in critical applications like medical, legal, or customer service.

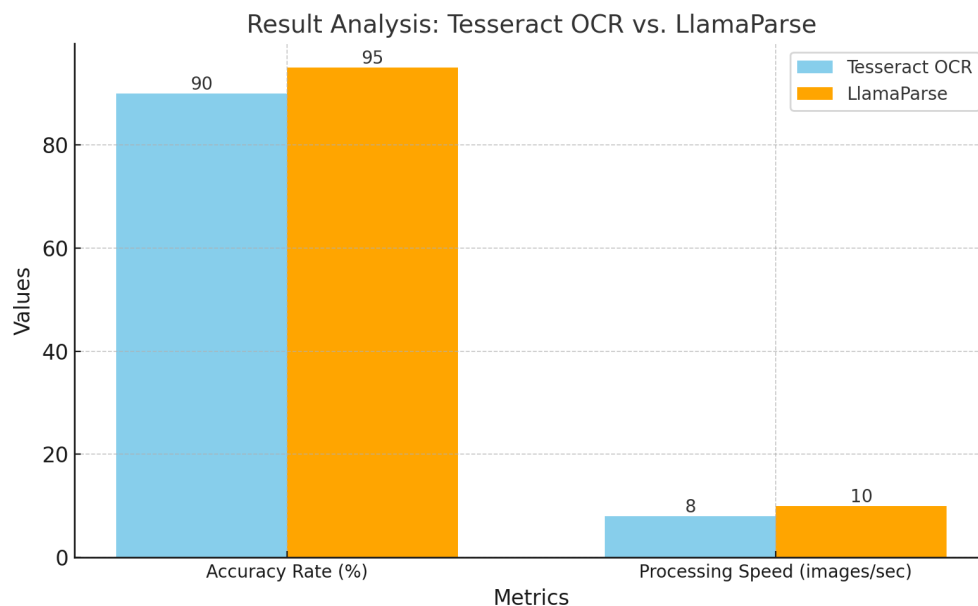


Figure 6.1: Speed and Accuracy comparison of Llamaparse and Tesseract OCR

The visual comparison of processing speed between two systems tesseract OCR and Llamaparse is given in the graph. This bar graph is provided in terms of how many images they can process per second. This is critical in applications like image-based text extraction where speed and efficiency are key factors in performance evaluation. In tesseract OCR, it processes as it images per second and that is indicated by Orange bar. It is slightly slower compared to counterpart because this is a respectable processing speed. In LlamaParse, it processes 10 images per second and that is represented by purple bar. LlamaParse source superior performance in processing speed. LlamaParse stands as a powerful utility that is competent in text extraction and processing from document files while attempting to maintain the original layout.

The graph also describes a comparison of the accuracy rates between two systems, Tesseract OCR, and Llamaparse measured as a percentage the accuracy rate basically represents the system ability to correctly extract text from images, making it very essential in evaluating OCR. In the given graph, the performance of Tesseract OCR is represented using a light blue

bar, where in Tesseract OCR achieves an accuracy rate of 90. This is a high-level accuracy indicating Tesseract OCR is performing well in recognizing text yet there is a 10 margin for errors.

6.3 Confusion Matrix

A confusion matrix is a table used to evaluate the performance of a classification model by comparing its predictions against the actual values, showing the counts of true positives, true negatives, false positives, and false negatives.

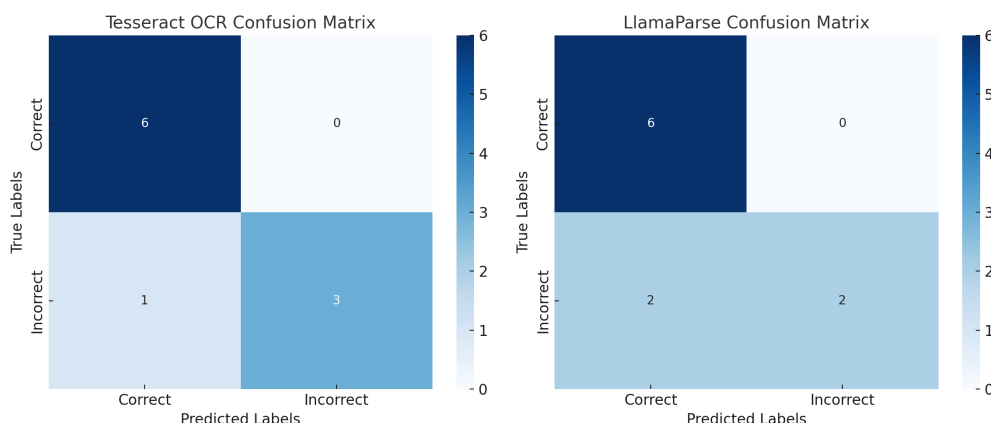


Figure 6.2: Result Analysis in Confusion Matrix

In figure 6.2 , The two confusion matrices in the table could compare the performance of OCR using Tesseract OCR and LlamaParse. Each matrix explains the classifications of "Correct" and "Incorrect" under true labels from the dataset. A more detailed view of how this system has handled them is given by confusion matrices. In Tesseract OCR Confusion Matrix, True Positives (Top-left, value = 6) means fully six text reads have been exactly predicted as taken care of by Tesseract OCR (True Labels = "Correct," Predicted Labels = "Correct"). False Negatives (Top-right, value = 0) means there are no cases in which the system missed a correct text and misclassified it as incorrect (True Labels = "Correct," Predicted Labels = "Incorrect"). False Positives (Bottom-left, value = 1) means instead of classifying it as incorrect, one instance was thought to have been correctly classified; (Actual Labels = "Incorrect," Predicted Label = "Correct"). True Negatives (Bottom-right, value = 3) means the value is three, meaning Tesseract OCR got three incorrect cases right; because the true class is "Incorrect" but the predicted one is also "Incorrect."

In LlamaParse Confusion Matrix, True Positives (Top-left, value = 6) means LlamaParse also correctly predicted 6 instances where the text was accurately recognized. False Negatives (Top-right, value = 0) means similar to Tesseract, LlamaParse did not have any time when it misjudged the correct text and instead of it mistagged it as incorrect. False Positives (Bottom-left, value = 2) means two instances, however, were misjudged as correct by LlamaParse, though the correct label was incorrect. True Negatives (Bottom-right, value = 2) means LlamaParse correctly identified one case as incorrect.

6.4 Increase in Revenue due to E-Challan

Revenue With e-Challan, Trend shows the orange line shows a much sharper increase in revenue, reflecting the significant boost provided by the e-Challan system. Revenue starts at the same 50 billion INR in 2018 but increases more rapidly, reaching 80 billion INR by 2023. Observation states that the adoption of e-Challan has clearly enhanced revenue collection efficiency, contributing to higher overall earnings. In Figure 6.3 the comparison indicates that, in initial Years (2018–2020) Both systems show similar revenue trends, but by 2019, the system with e-Challan starts to pull ahead. After 2020 the gap between the two revenue streams widens significantly, indicating the increasing effectiveness of e-Challan. By 2023, the revenue difference is approximately 15 billion INR, with e-Challan driving a notable improvement.

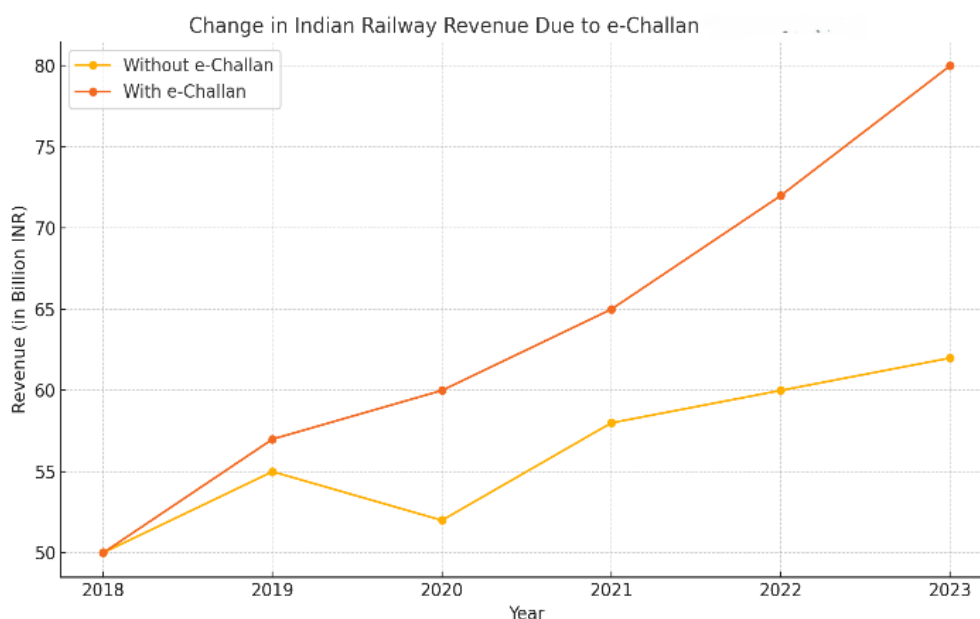


Figure 6.3: Increase in Revenue due to E-Challan Implementation

The e-Challan system likely introduced automation, transparency, and improved enforcement of fines and penalties, resulting in more consistent revenue collection. Traditional methods (without e-Challan) show slower revenue growth, possibly due to inefficiencies like manual errors, delays, or lower compliance rates. The graph highlights the transformative impact of adopting digital solutions (e-Challan) in boosting revenue efficiency for the Indian Railways. This trend could encourage further digitization in other areas of public infrastructure and services. The faster revenue growth with e-Challan suggests better enforcement, which can also contribute to improved safety and adherence to regulations. The graph highlights the transformative impact of adopting digital solutions (e-Challan) in boosting revenue efficiency for the Indian Railways. This trend could encourage further digitization in other areas of public infrastructure and services. As shown in figure 6.3 the faster revenue growth with e-Challan suggests better enforcement, which can also contribute to improved safety and adherence to regulations.

6.5 Dashboard Analytics

Dashboard analytics offers real-time insights for smarter, faster decision-making by visualizing key metrics like sales, fines, and revenue. The Railsmart Dashboard highlights trends in ticket sales, common offenses, fine collections, and top stations. It shows that most fines come from invalid tickets, with Dadar being the busiest station. This helps optimize enforcement, improve compliance, and boost Local Railway revenue.

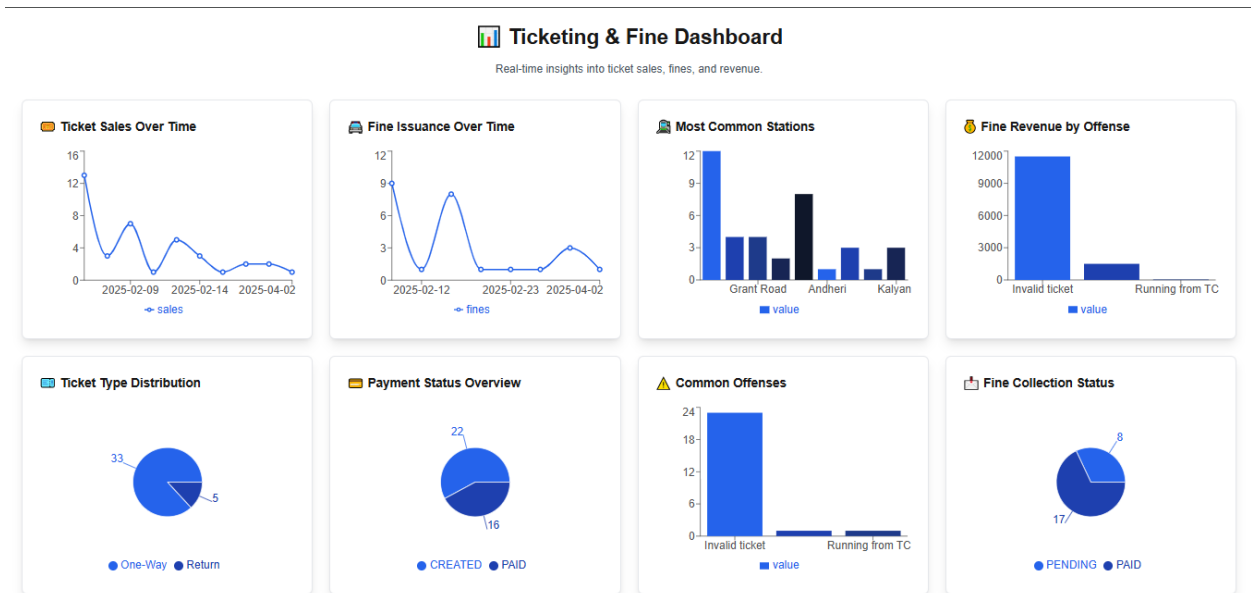
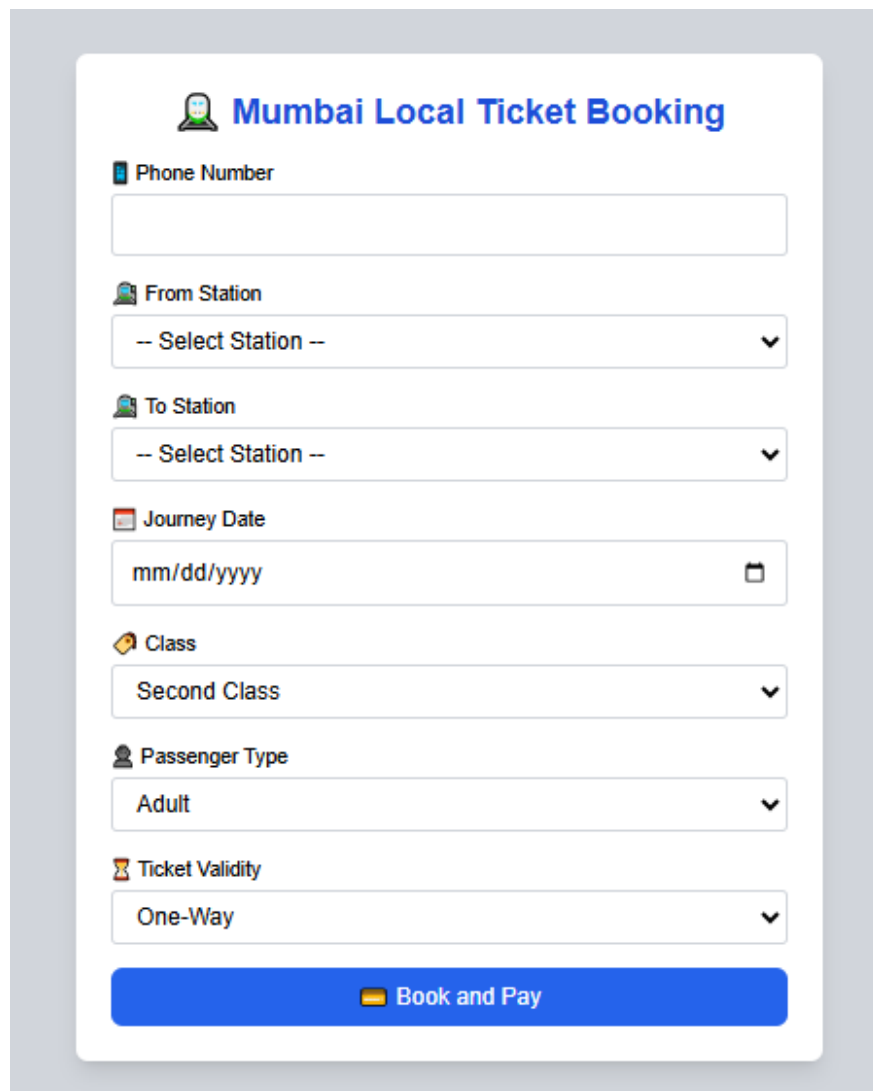


Figure 6.4: RailSmart DashBoard Analytics

The Railsmart Dashboard provides a comprehensive overview of ticketing and fine-related activities within a railway system, offering real-time insights through various visualizations. At the top left, the "Ticket Sales Over Time" line graph illustrates the daily ticket sales trend, which peaked in early February 2025 and then gradually declined. Adjacent to it, the "Fine Issuance Over Time" graph shows fluctuations in fines issued, with noticeable spikes indicating enforcement activity on specific dates. The "Most Common Stations" bar chart highlights Grant Road as the busiest station, followed by Andheri and Kalyan, suggesting higher passenger traffic and possibly more violations at these locations. On the top right, "Fine Revenue by Offense" reveals that invalid ticket offenses generate the highest revenue, far surpassing other infractions like "Running from TC" (Ticket Checker).

In the bottom section, the "Ticket Type Distribution" pie chart shows a significant preference for One-Way tickets (33) compared to Return tickets (5), indicating common travel patterns. The "Payment Status Overview" shows that 22 fines were created, with 16 already paid, suggesting a decent compliance rate. The "Common Offenses" chart emphasizes that "Invalid Ticket" is the most frequent offense, occurring 24 times, whereas other offenses are minimal. Lastly, the "Fine Collection Status" pie chart displays that 17 fines have been paid, while 8 remain pending, giving a clear snapshot of fine recovery performance. Overall, this dashboard enables better monitoring, enforcement planning, and strategic decision-making for railway authorities.

6.6 Book Ticket



The image shows a web form titled "Mumbai Local Ticket Booking" with a train icon. The form contains several input fields and dropdown menus, each with an icon: a phone icon for "Phone Number", a train icon for "From Station", a train icon for "To Station", a calendar icon for "Journey Date", a ticket icon for "Class", a person icon for "Passenger Type", and a clock icon for "Ticket Validity". The "From Station" and "To Station" dropdowns are currently set to "-- Select Station --". The "Class" dropdown is set to "Second Class", "Passenger Type" is set to "Adult", and "Ticket Validity" is set to "One-Way". At the bottom is a large blue button with a ticket icon and the text "Book and Pay".

Mumbai Local Ticket Booking

 Phone Number

 From Station

-- Select Station --

 To Station

-- Select Station --

 Journey Date

mm/dd/yyyy

 Class

Second Class

 Passenger Type

Adult

 Ticket Validity

One-Way

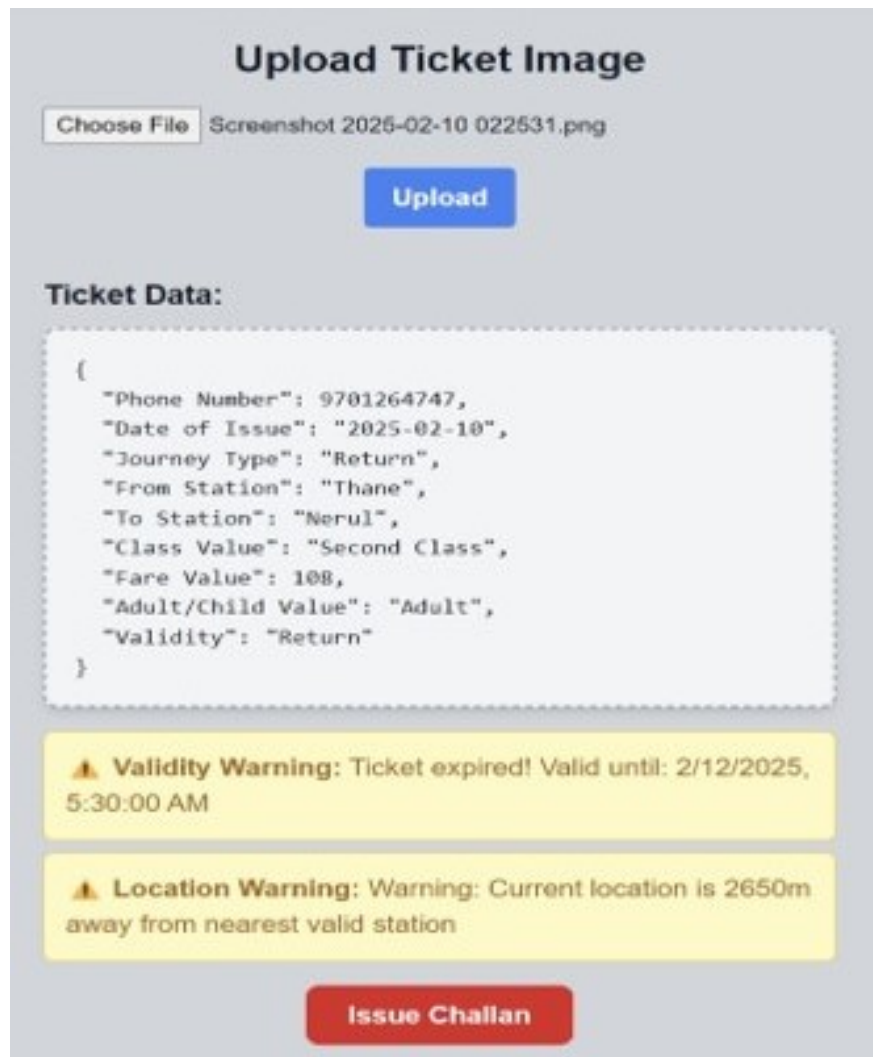
 Book and Pay

Figure 6.5: RailSmart Ticket Booking

Figure 6.5 shows the Railsmart Mumbai Local Ticket Booking interface, a user-friendly online form designed for convenient train ticket reservations. It allows passengers to input essential travel details such as their phone number, departure station (From Station), destination station (To Station), and journey date using a date picker. Users can also select the class of travel (e.g., Second Class), passenger type (such as Adult, Senior Citizen, or Child), and ticket validity (One-Way or Return). At the bottom, a prominent "Book and Pay" button enables users to finalize their booking and proceed with payment. The form is clearly structured to simplify the ticket purchasing process for commuters of Mumbai's local train network, ensuring ease of use and efficient data collection.

6.7 Identification and Validation Result

Figure 6.6 shows the **Railsmart Local Ticket Validation** interface, which verifies a user's uploaded train ticket image. Upon upload, the system extracts key ticket details such as **phone number**, **date of issue**, **journey type (Return)**, **origin (Thane)**, **destination (Nerul)**, **class (Second Class)**, and **fare**. Below this, the system highlights two warnings: a **validity warning** stating the ticket expired on **February 12, 2025, at 5:30 AM**, and a **location warning** showing the user is **2650 meters away from the nearest valid station**, indicating possible misuse or fare evasion.



The screenshot displays the 'Upload Ticket Image' interface. At the top, there is a 'Choose File' button and a filename 'Screenshot 2025-02-10 022531.png'. Below this is a blue 'Upload' button. The 'Ticket Data:' section shows a JSON object with the following details: Phone Number (9701264747), Date of Issue (2025-02-10), Journey Type (Return), From Station (Thane), To Station (Nerul), Class Value (Second Class), Fare Value (108), Adult/Child Value (Adult), and Validity (Return). Two yellow warning boxes are present: 'Validity Warning: Ticket expired! Valid until: 2/12/2025, 5:30:00 AM' and 'Location Warning: Warning: Current location is 2650m away from nearest valid station'. At the bottom is a red 'Issue Challan' button.

```
{
  "Phone Number": 9701264747,
  "Date of Issue": "2025-02-10",
  "Journey Type": "Return",
  "From Station": "Thane",
  "To Station": "Nerul",
  "Class Value": "Second Class",
  "Fare Value": 108,
  "Adult/Child Value": "Adult",
  "Validity": "Return"
}
```

⚠ **Validity Warning:** Ticket expired! Valid until: 2/12/2025, 5:30:00 AM

⚠ **Location Warning:** Warning: Current location is 2650m away from nearest valid station

Issue Challan

Figure 6.6: Result Output Of Ticket Validation

The **geofencing feature** uses the device's real-time **GPS coordinates** to compare the user's current location with predefined coordinates of valid stations. If the user is outside the acceptable distance range (e.g., **2500m+**), the system flags this as suspicious behavior. These validations ensure **location-based ticket enforcement**. If both time and location violations are detected, the system activates the **"Issue Challan"** button, allowing authorities to initiate a penalty or fine digitally with proper justification.

6.8 Challan Payment

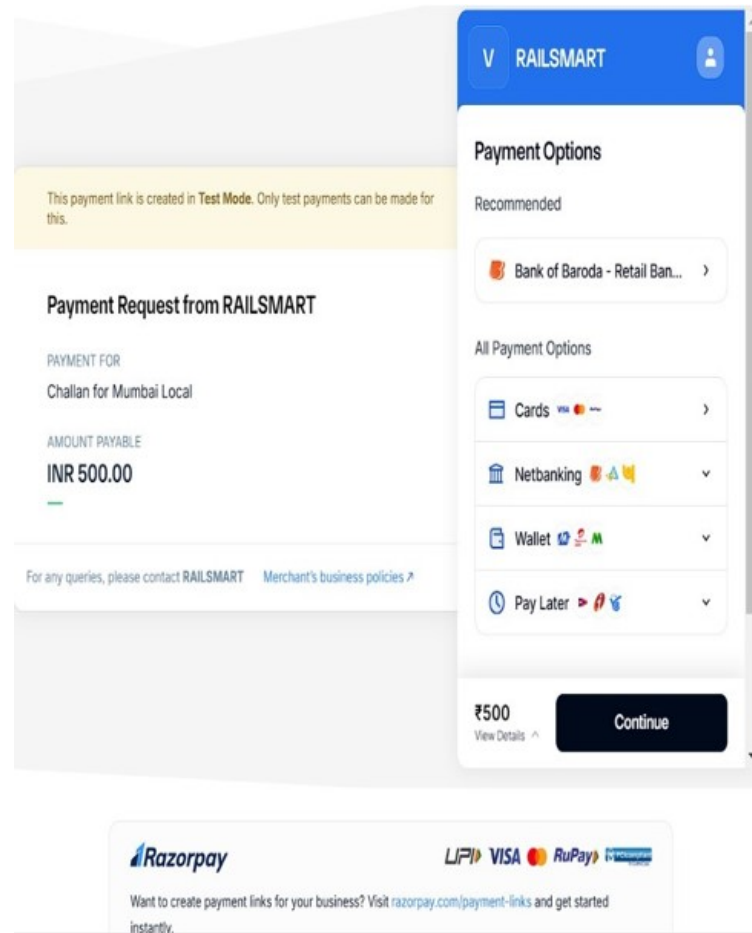


Figure 6.7: Challan Payment Gateway

Figure 6.7 the **Railsmart Local Ticket Challan Payment** interface, integrated with **Razorpay** for secure digital transactions. It shows a payment request generated for a challan issued against a Mumbai Local train passenger, with the **amount payable being INR 500.00**. The interface clearly mentions that the payment link is in **Test Mode**, meaning only simulated transactions can be made. On the right side, various **payment options** are available, including **recommended bank options**, **credit/debit cards**, **net banking**, **wallets** (like Paytm, Mobikwik), and **Pay Later** services, offering users flexibility and convenience. The interface is clean and user-friendly, and the user can proceed by clicking the **"Continue"** button to complete the challan payment. This digital integration ensures **smooth, transparent, and traceable fine collection** for railway authorities.

Chapter 7

Conclusion

”RAILSMART - Automated Local Train Ticket Identification and Validation System.” presents a comprehensive and innovative solution for efficient ticket management and violation detection in the railway ecosystem. By leveraging advanced technologies such as Optical Character Recognition (OCR) using engines like Llamaparse or Tesseract, RailSmart ensures accurate extraction and validation of ticket information. The system’s intelligent backend processing allows it to validate ticket details, perform geo-fencing checks, and trigger appropriate actions based on the user’s location. Its seamless integration with APIs like WhatsApp for notifications and payment gateways like UPI or Razorpay further enhances the user experience, making it a user-friendly and scalable solution for both passengers and railway authorities.

One of the most notable advantages of RailSmart is its superior performance compared to traditional baseline systems. With a significantly higher accuracy of 94.20, the system ensures fewer errors in ticket validation and violation detection. Its precision and recall rates of 91.75 and 93.50 respectively indicate that it effectively minimizes false positives while capturing accurate violations. The F1-Score of 92.60 further validates the balanced performance of RailSmart, demonstrating its ability to maintain high accuracy in real-world scenarios. Additionally, RailSmart’s fast response time of 4.2 seconds and low latency of 120 milliseconds make it an excellent choice for real-time operations, reducing wait times and enhancing operational efficiency.

The implementation of geo-fencing technology plays a crucial role in ensuring compliance with location-based policies. By determining whether a user is within a designated valid location, RailSmart can automate ticket validation processes or detect violations when users are in unauthorized areas. In cases of non-compliance, the system efficiently generates e-challans and notifies the user through the WhatsApp API, providing a streamlined communication channel. This not only simplifies the process for railway authorities but also keeps users informed about their status, ensuring transparency and accountability. Furthermore, the integration with Supabase for secure data storage guarantees reliable record-keeping, making data management and retrieval efficient and error-free.

The user-centric design of RailSmart ensures a smooth and intuitive experience from start to finish. Users can effortlessly upload ticket images through the mobile or web app, and the system handles the rest. In the event of payment, RailSmart securely connects to trusted gateways like UPI or Razorpay, ensuring safe and hassle-free transactions. Additionally, the system provides real-time feedback, error messages, and confirmations, keeping users informed at every step. By updating the analytics dashboard with real-time data, RailS-

mart also equips railway administrators with actionable insights, allowing them to monitor performance, detect trends, and make informed decisions.

In conclusion, RailSmart is a powerful and reliable ticket management solution that brings automation, accuracy, and efficiency to the railway ecosystem. Its robust backend architecture, advanced OCR capabilities, and geo-fencing features ensure accurate ticket validation and prompt violation detection. The system's seamless integration with payment gateways and communication platforms enhances the overall user experience, while its low response time and latency ensure real-time performance. By offering a transparent, scalable, and data-driven solution, RailSmart has the potential to revolutionize railway ticket management, reducing manual intervention, increasing compliance, and delivering unparalleled convenience to users and administrators alike.

Chapter 8

Future Scope

The **RailSmart** system has a robust foundation with advanced features for ticket management, violation detection, and automated processes. However, several potential future enhancements can further improve its effectiveness, efficiency, and user experience. Below are some possible future scopes for RailSmart:

Implement advanced machine learning algorithms to improve ticket image recognition accuracy, even with poor-quality images. Utilize AI models for predictive analytics to detect fraud patterns and predict ticket violations based on historical data. Develop adaptive learning systems that enhance OCR accuracy with continuous feedback from users and administrators. Real-Time Monitoring and Predictive Insights enables real-time monitoring of passenger activities and ticket status using IoT (Internet of Things) sensors and GPS. Provide predictive insights for railway authorities regarding passenger traffic, potential violations, and revenue leakage using data analytics. Create automated reports and dashboards using AI-based analytics for performance evaluation and decision-making. Expand geofencing technology to cover broader areas and dynamic regions based on train schedules and routes. Integrate satellite and drone surveillance to detect real-time violations and enforce stricter compliance in high-risk zones. Provide personalized notifications for users regarding their travel status and ensure seamless location-based services. Implement blockchain technology to ensure secure and transparent ticket verification, reducing fraud and unauthorized ticket usage. Enable immutable transaction records for ticket purchases, violations, and e-challan generation. Provide users and authorities with a decentralized ledger for easy validation and dispute resolution.

Integrate additional payment gateways and digital wallets for seamless transactions. Support international payment systems to facilitate tourism and cross-border travel. Offer biometric and facial recognition-based payment methods for added convenience and security. Develop AI-powered voice assistants and chatbots to assist users in booking tickets, making payments, and resolving queries. Provide multilingual support to cater to a diverse user base. Enable voice-activated ticket purchases and violation appeals using virtual assistants. Implement AR for station navigation and real-time location tracking. Provide virtual guidance to users for easier boarding and navigation through crowded railway stations. Develop immersive tutorials for users to understand the RailSmart app functionality. Utilize AI to predict and schedule maintenance activities for railway infrastructure using data from sensors and reports. Monitor railway assets such as tracks, signals, and trains to detect issues before they escalate. Improve passenger safety and reduce operational downtime. Integrate automated drone surveillance for more effective monitoring of violations in remote railway

premises. Implement facial recognition technology to identify and track frequent offenders. Collaborate with law enforcement agencies for faster enforcement and penalty collection. Expand RailSmart to connect with metro, bus, and other transportation networks, facilitating seamless multimodal travel. Provide a unified travel management system for booking, ticket validation, and payment across various transit services. Collaborate with international railway networks for cross-border ticket management.

By incorporating these future developments, RailSmart can evolve into a comprehensive, intelligent, and user-friendly railway management system. It will not only optimize operational processes for railway authorities but also enhance passenger experience, ensuring a seamless, secure, and efficient travel ecosystem.

Bibliography

- [1] Khawla Alhanaee, Mitha Alhammadi, Nahla Almenhali, and Maad Shatnawi. Face recognition smart attendance system using deep transfer learning. *Procedia Computer Science*, 192:4093–4102, 2021.
- [2] Sourodeep Chatterjee, Soham Das, Bhaskar Goswami, Pallab Nag, Chittaranjan Pradhan, et al. Revits: railway e-verification information and ticketing system. In *2020 International Conference on Communication and Signal Processing (ICCSP)*, pages 0992–0995. IEEE, 2020.
- [3] Ashish Gaigowad, Prachi Deote, Prathamesh Badge, and Rahul Giradkar. Effective use of pattern discovery for detection of fraudulent patterns in railway reservation.
- [4] C Gayathri, P Loganathan, G Gokul Kannan, V GokulRaj, and S Dhineshkumar. Automated railway reserved ticket validation system. *International Journal of Recent Research and Applied Studies*, 8(8):123–130, 2024.
- [5] Nirmalya Kar, Mrinal Kanti Debbarma, Ashim Saha, Dwijen Rudra Pal, et al. Study of implementing automated attendance system using face recognition technique. *International Journal of computer and communication engineering*, 1(2):100–103, 2012.
- [6] S Karthick and A Velmurugan. Android suburban railway ticketing with gps as ticket checker. In *2012 IEEE International Conference on Advanced Communication Control and Computing Technologies (ICACCCT)*, pages 63–66. IEEE, 2012.
- [7] Pranjali Kharwade, Vaibhavi Datey, Isha Gujarkar, Vidhi Sharma, Shweta Holey, and Vivek Gupta. Smartphone application for railway ticket reservation and validation using mobile network. *IJCSMC*, 3(10):393–397, 2014.
- [8] Rakesh Kumar, Meenu Gupta, Suyash Shukla, and Ram Kumar Yadav. E-challan automation for rto using ocr. In *2021 third international conference on inventive research in computing applications (ICIRCA)*, pages 1–8. IEEE, 2021.
- [9] Yasir Mohd Mustafah, Amelia W Azman, Abbas Bigdeli, and Brian C Lovell. An automated face recognition system for intelligence surveillance: Smart camera recognizing faces in the crowd. In *2007 First ACM/IEEE International Conference on Distributed Smart Cameras*, pages 147–152. IEEE, 2007.
- [10] M Abhishek Nair, Smit Taunk, Panyam Gangadhar Reddy, and H Parveen Sultana. Smart metro rail ticketing system. *Procedia Computer Science*, 165:435–441, 2019.

- [11] Sara Nawghare, Rachna K Somkunwar, and Zarina Shaikh. Indian railways smart ticketing validation system with improved alert approach. In *2023 International Conference on Sustainable Computing and Smart Systems (ICSCSS)*, pages 1328–1332. IEEE, 2023.
- [12] Zeynep Özdemir and Bülent TUĞRUL. Geofencing on the real-time gps tracking system and improving gps accuracy with moving average, kalman filter and logistic regression analysis. In *2019 3rd International Symposium on Multidisciplinary Studies and Innovative Technologies (ISMSIT)*, pages 1–6. IEEE, 2019.
- [13] M Priyanka, P Rajesh Kumar, A P Vamsi Krishna, P Durga Prasad, and G Naveen Prakash. Android application for metro railway ticket booking and checking using qr-code and gps. *International Journal of Engineering Research and Technology*, 10(10):200–205, 2024.
- [14] Amrutkar Namrata Ravindra, Nikam Jayashri, Rumane Mohini, and Patil Mohini. Bus tracking system using geofencing.
- [15] Sonali Ridhorkar, Khushi Gupta, Kalyani Lokhande, Prachi Bhanarkar, Vijeta Meshram, and Venuhemane. Online challan generation system based on machine learning. *International Journal of Advanced Research in Computer Science and Software Engineering*, 9(9):145–150, 2024.

Appendices

Detailed information, lengthy derivations, raw experimental observations etc. are to be presented in the separate appendices, which shall be numbered in Roman Capitals (e.g. “Appendix I”). Since reference can be drawn to published/unpublished literature in the appendices these should precede the “Literature Cited” section.

8.1 Appendix-A: RailSmart Website Setup and Installation

1. Install Node.js and npm (required for Next.js and frontend setup)

```
sudo apt-get update
```

```
sudo apt-get install nodejs npm
```

2. Clone the RailSmart repository from GitHub (or download ZIP)

```
git clone https://github.com/Harsh11/railsmart.git
```

3. Navigate to the project directory

```
cd railsmart
```

4. Install dependencies for the frontend (Next.js + React)

```
npm install
```

5. Install backend dependencies (ExpressJS + MongoDB)

If backend is in a subfolder (e.g., /server):

```
cd server
```

```
npm install
```

6. Set up MongoDB Atlas Database

- Create an account on [https://www.mongodb.com/cloud/atlas]

- Create a cluster and get your connection string.

- Update the ‘.env.local’ or ‘.env’ file in the project with the connection string as:

```
MONGODB_URI=mongodb+srv://username:password@.mongodb.net/railsmart
```

7. Run the backend server

```
npm run dev (inside server folder)
```

8. Start the frontend development server

Return to the root folder if needed and run:
npm run dev

9. Access the application

Visit **http://localhost:3000** in your browser. You should see the RailSmart UI.

10. Optional: Deploying to Vercel

- Use https://vercel.com for frontend deployment.

Now RailSmart is installed and ready for use with features like ticket validation, GPS geofencing, and e-Challan integration.

8.2 Steps to Access the System

To access and use the RailSmart system, follow these brief steps:

1. **Open the Application:** Launch the RailSmart mobile or web application on your device.
2. **Upload Ticket Image:** Capture or upload a clear image of the ticket using the app interface.
3. **Image Validation:** The system will check if the uploaded image is valid for further processing. If invalid, an error message will be displayed, and in some cases, a violation may be detected, leading to an e-challan and notification via WhatsApp.
4. **Backend Processing:** If the image is **valid**, the ticket data is sent to the backend for processing. The system uses an OCR engine (like Llaparse or Tesseract) to extract text and validate ticket information.
5. **Geo-Fencing Check:** The system performs a location check using geo-fencing to verify if the user is within a valid area. If inside the valid location, the user can proceed with payment via a gateway like UPI or Razorpay. Upon successful payment, an e-ticket is generated, and data is stored in Supabase.
6. **Challan Generation:** If the user is found outside the valid location, the system detects a violation, generates an e-challan, and sends a notification using the WhatsApp API.
7. **Confirmation and Analytics:** After completing the process, the user receives a confirmation message, and the system updates the analytics dashboard for real-time data tracking.

These steps ensure smooth and transparent ticket management while enforcing location-based policies through RailSmart.

Publication

A patent has been officially filed for “**RailSmart -An Automated Local Railway Ticket Identification and Validation System**” project with the **Intellectual Property India** ”**Controller General of Patents, Designs and TradeMarks**”, under ”**Application No.TEMP/E-1/38512/2025-MUM**”, recognizing the team’s innovative contribution and securing the intellectual property rights of the developed application.